

SansDIR: A Keyboard-Driven Terminal Application for Navigating, Inspecting, and Plotting Small-Angle Neutron Scattering Data

Changwoo Do

September 2026

Draft. Document has not been reviewed and approved for public release.

DOCUMENT AVAILABILITY

Online Access: US Department of Energy (DOE) reports produced after 1991 and a growing number of pre-1991 documents are available free via <https://www.osti.gov/>.

The public may also search the National Technical Information Service's [National Technical Reports Library \(NTRL\)](#) for reports not available in digital format.

DOE and DOE contractors should contact DOE's Office of Scientific and Technical Information (OSTI) for reports not currently available in digital format:

US Department of Energy
Office of Scientific and Technical Information
PO Box 62
Oak Ridge, TN 37831-0062
Telephone: (865) 576-8401
Fax: (865) 576-5728
Email: reports@osti.gov
Website: <https://www.osti.gov/>

This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

Neutron Scattering Division

**SANS DIR: A KEYBOARD-DRIVEN TERMINAL APPLICATION FOR
NAVIGATING, INSPECTING, AND PLOTTING
SMALL-ANGLE NEUTRON SCATTERING DATA**

Changwoo Do

September 2026

Prepared by
OAK RIDGE NATIONAL LABORATORY
Oak Ridge, TN 37831
managed by
UT-BATTELLE LLC
for the
US DEPARTMENT OF ENERGY
under contract DE-AC05-00OR22725

TABLE OF CONTENTS

LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
FOREWORD	vii
ABSTRACT	1
1. INTRODUCTION	1
1.1 MOTIVATION	1
1.2 DESIGN PRINCIPLES	2
1.3 HERITAGE AND RELATED TOOLS	2
1.4 SCOPE OF THIS REPORT	3
2. INSTALLATION AND DEPLOYMENT	4
2.1 ZERO-INSTALL DEPLOYMENT ON THE ANALYSIS CLUSTER	4
2.2 PORTABILITY OF THE LAUNCHER	4
2.3 DEPENDENCIES	5
2.4 VERIFYING AN INSTALLATION	5
3. SOFTWARE ARCHITECTURE	6
3.1 LAYERED STRUCTURE	6
3.2 THE COMMAND REGISTRY	6
3.3 DISPATCH PATHS	7
3.4 KEY BINDINGS AS DATA	7
3.5 THE COMMAND LINE PARSER	8
3.6 STARTUP BUDGET AND LAZY IMPORTS	8
3.7 CONCURRENCY MODEL	9
3.8 REPOSITORY LAYOUT	9
4. THE USER INTERFACE	10
4.1 SCREEN ANATOMY	10
4.2 PANES AND ACTIVATION	10
4.3 FILE LISTING AND COLOR CODING	11
4.4 TAGGING	11
4.5 COMMAND LINE AND HISTORY	11
4.6 HELP OVERLAY	11
5. NAVIGATION AND FILE OPERATIONS	12
5.1 NAVIGATION	12
5.2 FILE OPERATION SEMANTICS	12
5.3 REFRESH BEHAVIOR	13
5.4 ARCHIVING AND EMAIL	13
5.5 SHELL ESCAPE	13
6. PLOTTING	14
6.1 PLOTTING THE SELECTION	14
6.2 ONE-DIMENSIONAL PLOTS	14
6.3 TWO-DIMENSIONAL PLOTS	14
6.4 DETECTOR HEAT MAPS FROM NEXUS	15
6.5 GENERIC TABULAR PLOTS	15
7. METADATA INSPECTION AND EXTRACTION	18
7.1 SINGLE-FILE TREE BROWSER	18
7.2 KEYWORD SEARCH	18

7.3	BATCH EXTRACTION	18
7.4	OUTPUT MODES	18
7.5	A WORKED EXAMPLE	19
8.	CATALOG INTEGRATION	23
8.1	AUTHENTICATION	23
8.2	SEARCH MODEL AND CACHING	23
8.3	RUN CATALOG PANE	23
9.	DETECTOR MASK CREATION	24
9.1	ARCHITECTURAL CONSTRAINTS	24
9.2	INTERACTIVE EDITOR	24
9.3	COMMAND-LINE MASK CREATION	25
9.4	OUTPUT FORMATS AND REDUCTION COMPATIBILITY	25
10.	NON-INTERACTIVE COMMAND-LINE INTERFACE	26
10.1	LAUNCHING THE INTERFACE	26
10.2	BATCH METADATA EXTRACTION	26
10.3	MASK CREATION	26
11.	CONFIGURATION	27
12.	USANS REDUCTION	28
12.1	INSTRUMENT MODE	28
12.2	THE SETUP TABLE	28
12.3	INVOKING THE ENGINE	29
12.4	ONE KEY	29
12.5	A WORKED EXAMPLE	29
12.6	DESMEARING	30
13.	SUPPORTED DATA FORMATS	34
13.1	ASCII REDUCED DATA	34
13.2	NEXUS FILES	34
14.	TESTING AND QUALITY ASSURANCE	35
14.1	TEST SUITE	35
14.2	STATIC ANALYSIS AND STYLE	35
15.	PERFORMANCE	36
16.	SUMMARY	37
17.	REFERENCES	39
	APPENDIX A. COMMAND REGISTRY REFERENCE	A-1
	APPENDIX B. DEFAULT KEY MAP	B-1
	APPENDIX C. WORKED EXAMPLE SESSION	C-1

LIST OF FIGURES

Figure 1.	Layered architecture of sansdir.	6
Figure 2.	The help overlay, generated from the registry.	8
Figure 3.	The dual-pane interface.	10
Figure 4.	One-dimensional reduced data.	15
Figure 5.	Tiled two-dimensional plot of a temperature series.	16
Figure 6.	Detector heat map from a raw NeXus event file.	17
Figure 7.	Keyword search in the HDF5 tree browser.	19
Figure 8.	Batch metadata extraction (1 of 2).	21
Figure 9.	Batch metadata extraction (2 of 2).	22
Figure 10.	USANS reduction, end to end, on one key (1 of 2).	31
Figure 11.	USANS reduction, end to end, on one key (2 of 2).	32

LIST OF TABLES

Table 1.	Relocatable and fixed components of a shared installation	4
Table 2.	Runtime dependencies	5
Table 3.	Interfaces related to the command registry	7
Table 4.	Package structure	9
Table 5.	File-kind color coding in the panes	11
Table 6.	Navigation keys	12
Table 7.	Function-key file operations	12
Table 8.	What p draws for each kind of file	14
Table 9.	Interactive mask editor controls	24
Table 10.	Recognized ASCII column layouts	34
Table 11.	Performance measurement and regression budgets	36
Table A-1.	Registered commands. A trailing question mark marks an optional parameter	A-2
Table B-1.	Default key bindings	B-2

LIST OF ABBREVIATIONS

API	application programming interface
CI	continuous integration
CLI	command-line interface
CSV	comma-separated values
DAS	data acquisition system
DOE	US Department of Energy
DOI	digital object identifier
GUI	graphical user interface
HDF5	Hierarchical Data Format version 5
HFIR	High Flux Isotope Reactor
IPTS	Integrated Proposal Tracking System
JSON	JavaScript Object Notation
LLM	large language model
mc	Midnight Commander
NFS	Network File System
NIST	National Institute of Standards and Technology
OAuth2	Open Authorization version 2
ORNL	Oak Ridge National Laboratory
PNG	Portable Network Graphics
REST	representational state transfer
SANS	small-angle neutron scattering
SNS	Spallation Neutron Source
SSH	Secure Shell
TSV	tab-separated values
TTL	time to live
TUI	terminal user interface
USANS	ultra-small-angle neutron scattering
XML	Extensible Markup Language

FOREWORD

This report describes **sansdir**, a terminal application developed in the Neutron Scattering Division at Oak Ridge National Laboratory (ORNL). It supports the daily data-handling work of small-angle neutron scattering (SANS) instrument scientists and users. The software is used in production for Spallation Neutron Source (SNS) SANS workflows. It is installed across the analysis cluster and starts with the command **sansdir**.

The report serves users, developers, and reviewers. Section 1 gives the motivation, and Section 3 describes the software design. User procedures are given in Section 2 and Sections 4 through 12. Sections 13 through 16 document the data formats, tests, performance, and current limitations. The appendices give short reference tables for every command and key binding.

ABSTRACT

SANS users at the SNS spend a substantial part of their beam-time and post-experiment effort on tasks other than data reduction and analysis. They locate runs across Integrated Proposal Tracking System (IPTS) directories, inspect plots to check reduced data, read instrument logs from NeXus files, export selected metadata to tables, and package results for collaborators. These tasks are performed over Secure Shell (SSH) on the analysis cluster. In this environment, graphical user interface (GUI) tools that require X11 forwarding respond slowly.

This report describes **sansdir**, a keyboard-driven terminal user interface (TUI) with two panes. It places these tasks in one application and does not require a display server. The program follows the interaction model of the MDIR and Norton Commander file managers. It provides two independent directory panes, function-key actions, and file operations that move data from the active pane to the opposite pane by default. It also provides functions specific to SANS: automatic classification and plotting of reduced one- and two-dimensional data, detector heat maps calculated directly from raw NeXus event files without a Mantid runtime, a searchable Hierarchical Data Format version 5 (HDF5) tree browser, batch metadata extraction to comma-separated values (CSV) and tab-separated values (TSV), an interactive detector-mask editor with output compatible with the `drt_sans` reduction pipeline, and catalog search through the ORNL OnCat service. In ultra-small-angle neutron scattering (USANS) mode, it also prepares a setup table for review, invokes the established reduction engine, and provides truncated-Abel desmearing.

The main architectural decision is to define each interactive action as a typed command in one registry. Key bindings and the interactive command line call this registry. The non-interactive command-line interface (CLI) has a separate Click dispatch path but uses the same service functions. The registry also exports a tool schema that can support a future natural-language layer. The implementation contains approximately 18,500 lines of Python in 66 modules. The current test suite collects 805 tests. On an analysis node, the measured `sansdir -help` path has a median startup time of about 120 ms, within its 300 ms budget.

1. INTRODUCTION

1.1 MOTIVATION

A small-angle neutron scattering (SANS) experiment at the Spallation Neutron Source (SNS) produces one raw event-mode NeXus file for each run. These files are written to an Integrated Proposal Tracking System (IPTS) directory tree of the form `/SNS/EQSANS/IPTS-NNNNN/nexus/`. Reduction—performed by `drt_sans` (Oak Ridge National Laboratory 2025a) or by the automatic reduction service—emits one-dimensional $I(q)$ profiles, two-dimensional $I(q_x, q_y)$ arrays, transmission curves, and log files to a parallel `shared/` tree. One proposal routinely contains hundreds of runs and several thousand derived files.

Between measurement and publication, the instrument scientist and the user perform several repetitive tasks. Existing software does not connect these tasks well:

- **Navigation.** Move between IPTS directories, between the `nexus/` and `shared/` subtrees of one proposal, and between a proposal directory and a personal scratch area.
- **Triage.** Open a reduced $I(q)$ file to check whether a measurement worked, compare two or three curves on the same axes, or look at a raw detector image to confirm beam-center and beam-stop placement.

- **Log inspection.** Read sample environment values—temperature, shear rate, sample position—from the data acquisition system (DAS) logs stored inside a NeXus file, either for one run or as a table across many runs.
- **Packaging.** Select a subset of results, compress the files, and email them to a collaborator.

Each task is simple by itself. However, these tasks are repeated dozens of times each day from a terminal session on the analysis cluster, and each available tool addresses only one task. Mantid Workbench (Arnold, Bilheux, Borreguero, et al. 2014) is a full analysis environment and is heavy to launch only to inspect one curve. A file manager supports navigation but does not understand SANS file formats. Reading DAS logs requires a short `h5py` (Collette 2025) script. A graphical user interface (GUI) tool launched over Secure Shell (SSH) with X11 forwarding also has enough latency to make rapid iteration difficult.

The value of **sansdir** is not that it performs an impossible task. It places these repeated operations in one keyboard-driven application and reduces context switching. The application runs in the terminal that the scientist is already using.

1.2 DESIGN PRINCIPLES

Four principles guide the implementation. They also apply to future contributions.

1. **Speed first.** Every interaction should feel immediate. Blocking input/output is not allowed on the user-interface thread. Catalog queries, Hierarchical Data Format version 5 (HDF5) reads, and plot rendering run on worker threads or in separate processes.
2. **Keyboard-driven.** Common operations have a single-key or short-chord binding. Other registered actions are available from the interactive command line. Mouse support is acceptable but never required.
3. **Predictable.** A user familiar with MDIR, Norton Commander, or Midnight Commander (`mc`)-style file managers (The Midnight Commander Development Team 2025) should be able to recognize the interaction model. This requires two panes, function-key actions, and “operate from the active pane into the opposite pane” semantics.
4. **The command registry is the single source of truth for interactive actions.** Every interactive action is a registered command with a name, typed parameters, a description, and examples. Key bindings and the interactive command line dispatch through this registry. The command-line interface (CLI) uses a separate Click path to the same service functions. A key handler never calls business logic directly.

1.3 HERITAGE AND RELATED TOOLS

The interaction model comes from MDIR, a DOS-era file manager, and the Norton Commander family that includes `mc`. These file managers use two independent directory views, with exactly one active pane. Copy and move operations use “from the active pane to the other pane” as their default. This convention removes the source-and-destination dialog used by single-pane and graphical file managers. It is familiar to experienced Norton Commander users. The program therefore treats this behavior as a fixed requirement, not an aesthetic choice.

sansdir supports the existing SNS software stack and does not replace it. It does not perform native SANS reduction; `drt_sans` remains the reduction engine. For ultra-small-angle neutron scattering (USANS), it invokes the instrument team’s `usansred` engine but does not reimplement its reduction physics. The program does not perform model fitting or replace Mantid Workbench for interactive analysis. Its main

scope is the layer *around* reduction and analysis: finding files, viewing them, extracting metadata, and moving them. The detector-mask editor in Section 9 also produces an input for reduction, using the format accepted by `drtsans`.

An earlier tool by the same author, `eqsanscli` (Do 2025), provided command-line access to OnCat catalog search. The OnCat client in Section 8 uses the same application programming interface (API) endpoints and authentication flow. However, the client was written for this program and did not copy the `eqsanscli` source code.

1.4 SCOPE OF THIS REPORT

This report describes version 0.10.2. Section 2 describes installation and deployment. Section 3 presents the software architecture, with emphasis on the command registry. Sections 4 through 9 describe the user-facing features. Section 10 describes non-interactive use, and Section 11 describes the configuration file. Section 12 describes reduction setup and desmearing for USANS. Sections 13 through 15 describe the supported data formats, test suite, and measured performance. Section 16 summarizes the work and its limitations.

2. INSTALLATION AND DEPLOYMENT

2.1 ZERO-INSTALL DEPLOYMENT ON THE ANALYSIS CLUSTER

The main deployment target is the Oak Ridge National Laboratory (ORNL) analysis cluster. The program is installed across the cluster as the command `sansdir`. This is a released, non-editable installation with its own Python environment, so users do not need to install it. The program can be started from any terminal on the cluster: a login node, a web-terminal session, or an SSH connection:

```
sansdir
sansdir /SNS/EQSANS/IPTS-12345/shared
```

The first form opens the interface in the current working directory. The optional argument gives the starting path for the left pane. This allows the user to open a proposal's `shared` folder directly. The right pane starts in the same directory. Any readable directory is accepted. A path under `/SNS/USANS` also starts the program in USANS mode (Section 12).

If `sansdir` is not on `PATH` on a particular machine, the same launcher is available at `/SNS/EQSANS/shared/bin/sansdir`.

2.2 PORTABILITY OF THE LAUNCHER

The installation contains two launch scripts with different behavior. The Python console script at `.venv/bin/sansdir` has an *absolute* interpreter path in its shebang. This script can be copied or symbolically linked to another location. The kernel still uses the bundled Python. Its execution chain is:

```
[copy or symlink of <install>/.venv/bin/sansdir]
-> <install>/.venv/bin/python      (via absolute shebang)
-> the installed sansdir package  (inside that environment)
```

The shell wrapper at `bin/sansdir` is different. It resolves the real location of the wrapper and looks for `../.venv` relative to that path. It can be run in place or reached through a symbolic link, but copying the wrapper alone to another directory breaks this relative lookup.

Neither launcher uses the user's active conda environment, current working directory, or the first Python on `PATH`. A symbolic link is normally preferred. When the shared installation is replaced, the link follows the current launcher, while a copied console script can continue to point to an old environment.

Table 1 shows which components must remain on the shared mount and which can be moved.

Table 1. Relocatable and fixed components of a shared installation

Path	Status	Note
<code>.venv/</code>	Fixed	Bundled Python interpreter, dependencies, and installed release.
<code>.venv/bin/sansdir</code>	Copy or link	Python entry point with an absolute shebang.
<code>bin/sansdir</code>	Link only	Shell wrapper; locates <code>../.venv</code> from its resolved source location.

2.3 DEPENDENCIES

Python 3.10 or newer is required. Table 2 lists the runtime dependencies and their roles. The dependency set is kept small. A proposal to add a library—especially `pandas`, `scipy`, or a character-cell plotting library—must be justified against the startup budget in Section 3.6.

Table 2. Runtime dependencies

Package	Minimum	Role
<code>textual</code>	0.80	Terminal user interface (TUI) framework; asynchronous, themeable (McGugan 2025).
<code>click</code>	8.1	CLI subcommand dispatch (Ronacher and The Pallets Team 2025).
<code>matplotlib</code>	3.7	All plotting (Hunter 2007).
<code>numpy</code>	1.24	Array input/output for ASCII and NeXus data (Harris, Millman, Walt, et al. 2020).
<code>h5py</code>	3.9	NeXus and HDF5 access (Collette 2025).
<code>httpx</code>	0.27	Asynchronous representational state transfer (REST) client for OnCat.
<code>send2trash</code>	1.8	Recoverable delete, with a fallback for cluster filesystems that provide no trash directory.
<code>tomli</code>	2.0	Tom’s Obvious Minimal Language parsing on Python 3.10 only; Python 3.11 and newer use the standard-library <code>tomllib</code> .

Note: Optional extras are `qt` (installs `PyQt5` for an interactive `matplotlib` backend), `tk` (a marker extra; the `tkinter` module ships with most Python builds), `llm` (the Anthropic client for the planned natural-language layer), and `dev`.

2.4 VERIFYING AN INSTALLATION

```
sansdir --version      # prints the version and exits
sansdir --help         # subcommands and worked examples
sansdir extract --help # per-subcommand help
```

A successful `--version` command confirms that the interpreter, package, and entry point are resolved correctly. This command does not import `Textual`, `matplotlib`, or `h5py`. Therefore, it also checks the lazy-import design described in Section 3.6.

3. SOFTWARE ARCHITECTURE

3.1 LAYERED STRUCTURE

Figure 1 summarizes the main call paths. Interactive key bindings and the `:` line convert a user action into a command name and arguments, then use the command registry. The non-interactive CLI uses a separate Click path. Both paths use reusable modules for files, metadata, plotting, masks, and USANS. External resources are shown at the bottom.

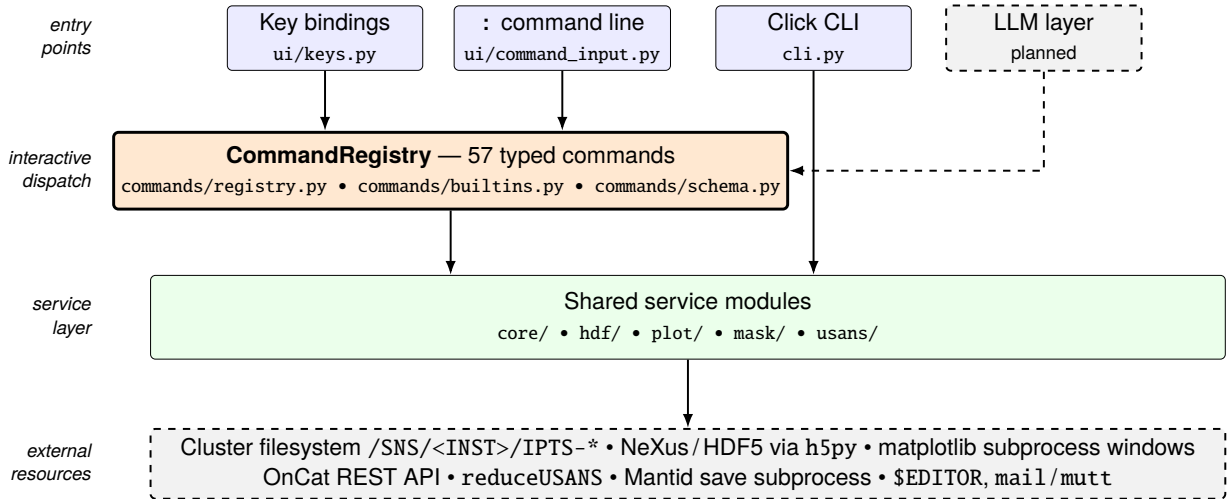


Figure 1. Layered architecture of sansdir. Key bindings and the interactive command line use one command registry. The CLI uses a separate Click path to the same service functions, and the natural-language layer is planned but not implemented. Reusable data operations are kept in service modules, while the application and interface modules coordinate the current session and views.

The Textual application object keeps presentation and session state, including the active pane, cursor and tags, and the modal screen stack. Interface modules also use several lightweight helpers directly for file listings, history, and read-only HDF views. The larger data operations are kept in reusable modules so that they can be tested without creating a terminal application.

3.2 THE COMMAND REGISTRY

The registry is the main design decision for the interactive interface. Each interactive action—change directory, tag, copy, plot, extract metadata, search the catalog, create a mask—is a `Command` object. The object contains a dotted name, a one-sentence description, a tuple of typed parameters, a handler, optional aliases and examples, and a `danger` flag for destructive operations. Each parameter is also a typed object. It has a name, a type from a fixed vocabulary (`path`, `glob`, `string`, `int`, `float`, `bool`, `enum`, `files`), a description, a required flag, a default, and—for enumerations—a list of permitted values.

Validation occurs when a command is constructed and when it is dispatched. A parameter with an unknown type, or an enumeration without choices, raises an error when the command is built. Therefore, a malformed command definition fails during import instead of when a user first presses the key. During dispatch, an unknown keyword argument or a missing required argument raises `TypeError` before the handler runs.

Handlers can be synchronous or asynchronous. `dispatch` is asynchronous and awaits the result when the handler is awaitable. Thus, callers use one calling convention for both types of handlers. A synchronous convenience wrapper is provided for call sites outside an event loop. It raises an error if it is called from inside an event loop.

Version 0.10.2 registers 57 commands. Appendix A lists them in full.

3.3 DISPATCH PATHS

Table 3 shows the two implemented callers of the registry and two related interfaces. Key bindings and the interactive command line dispatch through the registry. The CLI uses the same service modules through a separate Click path, and the large language model (LLM) layer is planned. Some interface modules also call read-only helpers directly when they build or update a view.

Table 3. Interfaces related to the command registry

Caller	Path
Key binding	<code>ui/keys.py</code> maps F6 to <code>ui.copy_tagged</code> , resolves the active pane’s selection and the inactive pane’s directory into keyword arguments, and dispatches.
Command line	<code>ui/command_input.py</code> parses <code>cd /SNS/EQSANS</code> into the command name <code>nav.cd</code> and the argument <code>path</code> , then dispatches.
CLI	<code>sansdir extract -k ...</code> builds arguments in a separate Click dispatch path and calls the same extraction service used by the interactive dialog.
LLM layer	Planned, not implemented. The registry already exports a JavaScript Object Notation (JSON) tool schema; a natural-language front end would emit command names with typed arguments, display them for confirmation, and dispatch.

This design has three practical benefits. First, key bindings and the interactive command line use the same commands, while the CLI reuses the same underlying service functions. Second, the help overlay is generated from registry metadata instead of being maintained by hand. Figure 2 shows this generated output. Third, the exported schema provides a defined input for a future natural-language caller.

3.4 KEY BINDINGS AS DATA

The key map is a list of immutable `KeyBinding` records. Each record has a key, a command, a description, and an optional *argument resolver*. The resolver is a small named function that converts the application state into the keyword arguments required by the command. For example, the resolver for `m` obtains the path under the cursor in the active pane and returns it as the `path` argument for `hdf.show_keys`.

Python can define a one-line helper function without a name (a *lambda*), but this function appears only as `<lambda>` in an error message. Each resolver is therefore an ordinary function with a descriptive name. The error traceback and help overlay can then identify the resolver. The default map has 48 bindings, and 38 of them are shown in the help overlay. The remaining bindings are duplicate encodings of the same key. These duplicates are needed because terminals report some keys differently. For example, most terminals report the equals sign as `equals_sign`, while others report `=`. Both forms are registered, but the duplicate entry is hidden from help.

A test checks that every binding names a command in the fully bound registry. Therefore, a typo in the key map fails in continuous integration (CI) instead of at runtime.

Users can replace or add bindings through the `[keys]` section of the configuration file. An entry with an unknown command is removed at startup with a notification, instead of stopping the launch.

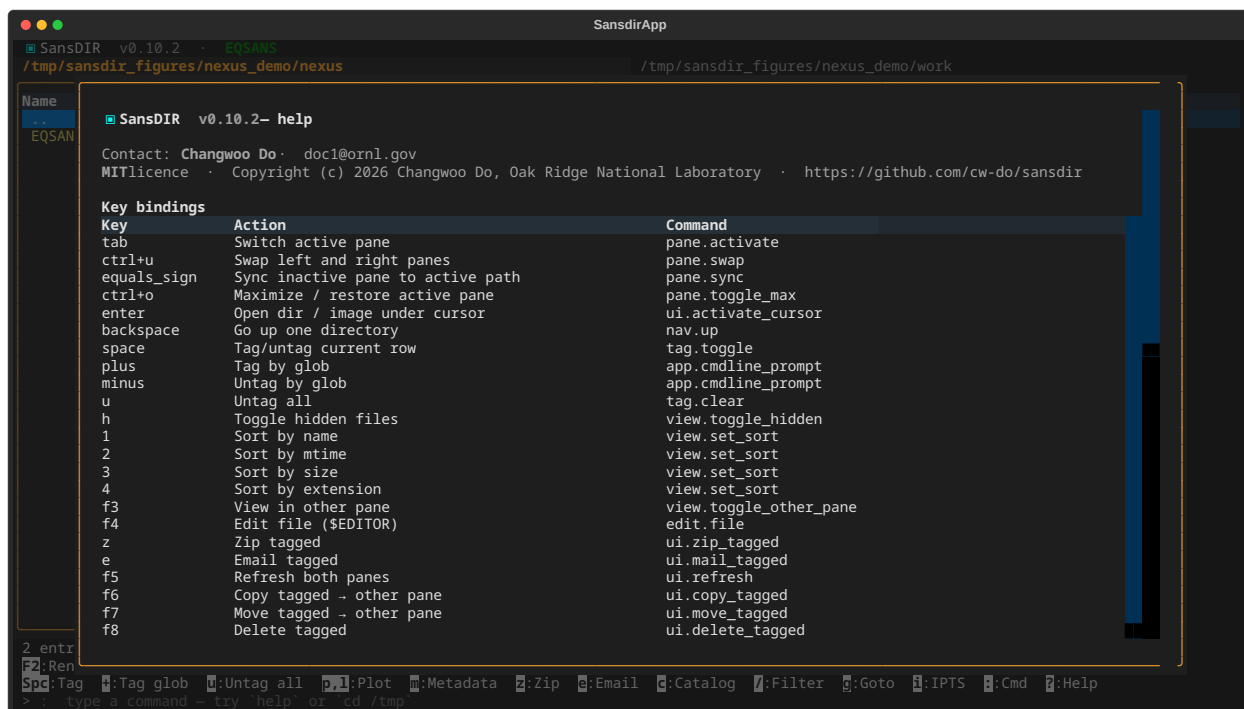


Figure 2. The help overlay, generated from the registry. Each row shows a keystroke, its readable action, and the underlying command name. The third column shows the connection between the two interfaces. A key action can also be entered as a typed command. A compatible command that needs no unresolved arguments can be assigned to a key.

3.5 THE COMMAND LINE PARSER

The `:` line accepts registered command names and aliases with positional and keyword arguments. It does not interpret natural language and does not use `eval` or `exec`. This is a safety boundary. Natural-language input is reserved for the future LLM layer, which can produce only registry-defined commands with typed arguments.

Aliases keep the typed interface short. `nav.cd` is available as `cd`, `hdf.batch_extract` as `extract`, `oncat.search` as `ipts`, and `shell.run` as `!`. Command history persists across sessions.

3.6 STARTUP BUDGET AND LAZY IMPORTS

The startup budget is 300 ms. The measured value in this report is for `sansdir -help`, not for the first paint of the terminal user interface (TUI). This path was measured again after USANS reduction and desmearing were added. Its median end-to-end wall-clock time on an analysis node is about 120 ms. About 40 ms is used to start the Python interpreter, and about 40 ms is used to import the command module. The best case is near 60 ms. The new features did not increase this value because the help path does not import them. A check confirms that `numpy`, `h5py`, `matplotlib`, `Textual`, and `httpx` are absent from `sys.modules` after `import sansdir.cli`. The TUI first-paint time has not been measured for this report.

The CLI imports only `click` and the version constant at module scope. `Textual`, `matplotlib`, `h5py`, `httpx`, and the registry are imported inside the subcommands that need them. Therefore, the commands `sansdir version` and `sansdir -help` do not load the scientific stack. The TUI loads `Textual` at startup but delays `matplotlib` and `h5py` until the first plot or metadata operation.

The `-help` startup time is checked after a change to the import graph.

3.7 CONCURRENCY MODEL

Three mechanisms keep the interface responsive:

- **Thread offload.** Blocking library calls, mainly `h5py` reads, run through `asyncio.to_thread`. The HDF5 key search in Section 7.2 is an example. Walking through a large raw NeXus file takes seconds, so the walk runs on a thread. The result is stored in the screen object for later queries.
- **Textual workers.** Longer interactive operations run as background workers, while the event loop continues to process keystrokes. For example, the mask editor launcher waits for the editor subprocess in a worker and refreshes both panes when the editor saves and exits.
- **Subprocesses.** Plot windows run in a separate process. This is the only reliable way to give matplotlib its own GUI event loop without competing with the Textual event loop.

3.8 REPOSITORY LAYOUT

Table 4 lists the package structure. The implementation contains approximately 18,500 lines in 66 Python modules.

Table 4. Package structure

Path	Contents
<code>cli.py</code>	Click entry point; <code>tui</code> , <code>extract</code> , <code>mask</code> , <code>usans</code> , and <code>version</code> subcommands.
<code>app.py</code>	The Textual application shell.
<code>config.py</code>	Configuration loader for Tom’s Obvious Minimal Language (Preston-Werner et al. 2021) (“Tom’s Obvious Minimal Language,” the format’s real name), read with the standard-library <code>tomllib</code> .
<code>commands/</code>	<code>registry.py</code> (the Command model and dispatch), <code>builtins.py</code> (56 app-bound registrations plus <code>app.quit</code> , 57 total), <code>parser.py</code> (the <code>:</code> line), <code>schema.py</code> (JavaScript Object Notation (JSON) tool schema export).
<code>core/</code>	<code>filesystem.py</code> (listing and sorting), <code>fileops.py</code> (copy, move, delete), <code>archive.py</code> , <code>mailer.py</code> , <code>oncat.py</code> , <code>history.py</code> , <code>instrument.py</code> (SANS/USANS mode resolution).
<code>hdf/</code>	<code>reader.py</code> (safe open, tree walk, value previews), <code>metadata.py</code> , <code>batch.py</code> (parallel extraction).
<code>plot/</code>	<code>detect.py</code> (file-kind classification), <code>ascii1d.py</code> , <code>ascii2d.py</code> , <code>tile.py</code> , <code>generic.py</code> , <code>image.py</code> , <code>hdf5_detector.py</code> , <code>backend.py</code> , <code>window.py</code> (the plot subprocess).
<code>usans/</code>	<code>grouping.py</code> (title blocks), <code>reconcile.py</code> (scan counts derived from disk), <code>table.py</code> (the setup comma-separated values (CSV)), <code>catalog.py</code> (table and notes), <code>runner.py</code> (invoking the external engine), <code>desmear.py</code> (truncated Abel inversion) and <code>desmear_io.py</code> (reading and writing curves). Pure; imports no interface or network module.
<code>mask/</code>	<code>core.py</code> (shapes and the mask builder), <code>detector.py</code> , <code>banktube.py</code> , <code>writers.py</code> , <code>mantid_writer.py</code> , <code>gui.py</code> , <code>api.py</code> .
<code>ui/</code>	<code>panel.py</code> , <code>pane_slot.py</code> , <code>keys.py</code> , <code>command_input.py</code> , <code>dialogs.py</code> , <code>hdf_tree.py</code> , <code>run_catalog.py</code> , <code>oncat_browser.py</code> , <code>help.py</code> , <code>statusbar.py</code> , <code>titlebar.py</code> , <code>pathbar.py</code> , <code>key_hint_bar.py</code> , <code>inline_viewer.py</code> , <code>prompt_panel.py</code> .

4. THE USER INTERFACE

4.1 SCREEN ANATOMY

Figure 3 shows a running session. From top to bottom, the screen contains a title bar with the program name and version; a two-part path bar that shows the working directory of each pane and highlights the active path; the two file panes; a status line with the entry count and number of tagged files; a two-row key hint bar in the Norton Commander style; and the command line.

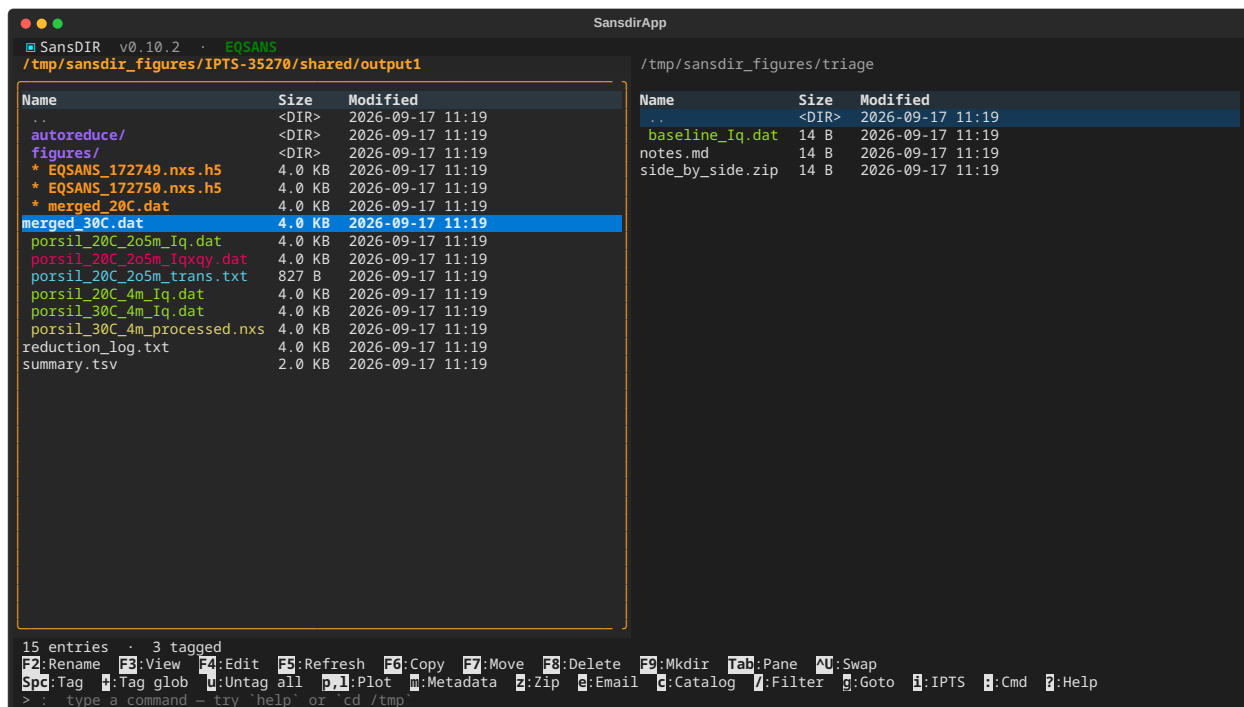


Figure 3. The dual-pane interface. The highlighted border and path show that the left pane is active. Three files are tagged. They have a leading asterisk and bold text, and the status line gives the count. Color identifies the file kind. Directories, one-dimensional reduced data, two-dimensional reduced data, transmission files, and NeXus files each have a different color.

The path bar displays the /gpfs/neutronsfs/instruments prefix as /SNS. Users normally write and recognize the location in this form. The two paths refer to the same mount, so this display does not change the location.

4.2 PANES AND ACTIVATION

The two panes are instances of the same widget class, but they have independent states: working directory, cursor position, tag set, sort key, and filter. One pane is active at a time and is identified by its border color. **Tab** changes the active pane.

Several commands act on both panes. **Ctrl+U** swaps their contents and quickly reverses the direction of a copy. **=** changes the inactive pane to the directory of the active pane. This is the usual preparation for an operation in the same directory. **Ctrl+O** expands the active pane to full width, and a second press restores both panes. Full width is a toggle and not the default because the program is designed around two panes.

The run catalog described in Section 8 always appears in the right pane, independent of which pane was active when it was requested. This keeps the layout consistent. The catalog remains on the right, and **Ctrl+U** swaps file panes without changing it.

4.3 FILE LISTING AND COLOR CODING

Each pane lists name, size, and modification time. Directories sort first. Sorting is by name, modification time, size, or extension, selected with the number keys **1** through **4**. Hidden files are suppressed by default and toggled with **h**. Symbolic links are followed but flagged.

Color identifies the file kind and helps the user scan a directory quickly (Table 5).

Table 5. File-kind color coding in the panes

Kind	Rendering
Directory	Bold blue
Symbolic link	Cyan
Iq.dat, one-dimensional reduced	Green
Iqxqy.dat, two-dimensional reduced	Magenta
trans.txt, transmission	Cyan
*.nxs.h5 and *.nxs, NeXus	Bright yellow
Executable	Bold red
Tagged row	Bold yellow with a leading asterisk

4.4 TAGGING

Tags select files for every multi-file operation. **Space** changes the tag on the row under the cursor. **+** asks for a glob and tags each visible entry that matches it. **-** removes tags by glob, and **u** removes all tags in the active pane. Each pane has its own tags, which are cleared when the pane changes directory.

All operations follow the same rule. If the active pane has tagged files, the operation acts on them. Otherwise, it acts on the file under the cursor. Thus, a single-file operation does not require a tag, and a multi-file operation does not require a separate “select mode.”

/ filters the active pane by substring. It can be used with glob tagging: filter to a subset, tag all visible files, and then perform the operation.

4.5 COMMAND LINE AND HISTORY

The command line is always visible at the bottom of the screen. **:** moves the focus to it. It accepts any registered command name or alias. Some shortcut keys fill the command line instead of immediately performing an action. **+** opens it with **tag.glob**, **g** with **cd**, and **F9** with **mkdir**. Therefore, the connection between a key and its command is visible during use and not only in the help overlay.

History persists across sessions in the cache directory.

4.6 HELP OVERLAY

? opens the help overlay in Figure 2. The program generates it from the registry and key map at runtime. Therefore, the overlay includes user key rebindings and remains consistent with the code.

5. NAVIGATION AND FILE OPERATIONS

5.1 NAVIGATION

Table 6 lists the navigation keys. The action of **Enter** depends on the selected item. It opens a directory, opens an image viewer, previews a text file in the other pane, or plots the run under the cursor in the run catalog.

Table 6. Navigation keys

Key	Action
Tab	Switch the active pane
Arrow keys, j k	Move the cursor
Enter	Descend into a directory, open an image, preview text, or plot a catalog row
Backspace	Ascend one directory
g	Prompt for a path
G	Full-screen directory-tree browser
/	Filter the active pane by substring
=	Synchronize the inactive pane to the active pane's directory
Ctrl+U	Swap the two panes
Ctrl+O	Maximize or restore the active pane

5.2 FILE OPERATION SEMANTICS

File operations follow the Norton Commander convention. They use the selection in the active pane, and the working directory of the *inactive* pane is the default destination. Table 7 lists the function-key assignments.

Table 7. Function-key file operations

Key	Operation	Behavior
F2	Rename	In-place rename of the file under the cursor.
F3	View	Read-only view in the opposite pane; Esc or a second F3 closes it.
F4	Edit	Opens \$EDITOR.
F5	Refresh	Re-reads both directory listings.
F6	Copy	Selection into the inactive pane, after confirmation.
F7	Move	As copy, then removes the source.
F8	Delete	Always confirms; uses <code>send2trash</code> where available.
F9	Make directory	Prompts in the active pane.
c	Catalog toggle	Flips the inactive pane between file list and run catalog. ^a

^a The catalog toggle is bound to **c** rather than **F10** because most terminal emulators intercept **F10** for their own menu bar. The classic copy, move, and delete bindings are correspondingly shifted from **F5–F7** to **F6–F8** to make room for refresh.

Two behaviors were added after user feedback. After a file is deleted, the cursor remains near its previous position. It stays on the replacement row, or on the new last row if the deleted file was last. This makes it easier to remove several files individually. Also, every destructive operation requires confirmation and is logged.

5.3 REFRESH BEHAVIOR

Operations performed in the application, including copy, move, rename, directory creation, batch extraction, and archive creation, refresh the affected panes automatically. **F5** provides a manual refresh for external changes. These can come from another shell session, an autoreduction job writing new output, or delayed Network File System (NFS) attribute updates.

The mask editor runs as a separate subprocess so that the interface remains responsive. A worker waits for its exit code. If the editor reports that a file was saved, both panes are refreshed and the new mask file appears.

5.4 ARCHIVING AND EMAIL

z creates a zip archive from the selection. It asks for the archive name and uses the inactive pane's directory by default. The command line also provides `archive.tar_gz` for a gzip-compressed tar archive. **e** opens a dialog and sends the selection as attachments through the configured `mail` or `mutt` program.

5.5 SHELL ESCAPE

`shell.run`, with the alias `!`, suspends the interface and runs a shell command. The interface returns when the command finishes. This provides access to operations that are not implemented in the program.

6. PLOTTING

6.1 PLOTTING THE SELECTION

p plots the active selection. It uses the tagged files when tags are present and otherwise uses the file under the cursor. The program determines the plot from the file, so the same key works for each supported type of SANS data. Table 8 lists the resulting plots.

Table 8. What **p draws for each kind of file**

Selection	Result
Iq.dat, 2–4 columns	Logarithmic $I(q)$ plot, with error bars when a σ_I column is present. Several files overlay on shared axes.
trans.txt	Transmission $T(\lambda)$ on linear axes, λ in Å.
Iqxqy.dat, 4 or 6 columns	Two-dimensional heat map; several files tile into one window.
*.nxs.h5 raw event file	Detector heat map of summed counts.
*.nxs processed Mantid file	Detector heat map through the same view.
Any other tabular file (1)	Linear plot using the header row as axis labels.

Plots open in separate windows, and the terminal remains available while the figures are open. Several figures can be shown at the same time. On a host without a display, the figure is written as a Portable Network Graphics (PNG) under `~/.cache/sansdir/plots/`, and the status bar reports its path. Thus, the same key also works in a plain SSH session without forwarding.

Classification does not use only the file extension. Mantid writes processed output as `.nxs` without the `.h5` suffix, and reduced files do not always have predictable names. A file containing HDF5 data is treated as a NeXus file regardless of its name. Column counts and header keywords distinguish the ASCII formats.

6.2 ONE-DIMENSIONAL PLOTS

Reduced $I(q)$ files use logarithmic axes by default and include error bars when a third column is present. Several tagged files are overlaid on shared axes with a filename legend. This supports common comparisons such as one sample at two temperatures or one measurement at two detector distances. Figure 4 shows an example with one curve.

Transmission files use linear axes, with wavelength in Å on the abscissa and transmission on the ordinate. The labels and title are set accordingly.

6.3 TWO-DIMENSIONAL PLOTS

Four- and six-column $I(q_x, q_y)$ files are drawn as heat maps on q_x and q_y axes. A logarithmic color scale is used by default.

When several two-dimensional files are tagged, they are tiled in one window and each subplot uses its filename as the title. Two color-bar modes are available. In *shared* mode, one scale is used for all panels and their intensities can be compared directly. In *independent* mode, each panel has its own scale, which can make weak patterns more visible. Shared mode is the default because the panels are normally being compared. The standing preference is set with `[ui].colorbar_mode`. One plot can override it with `:plot.iqxqy colorbar_mode=independent`.

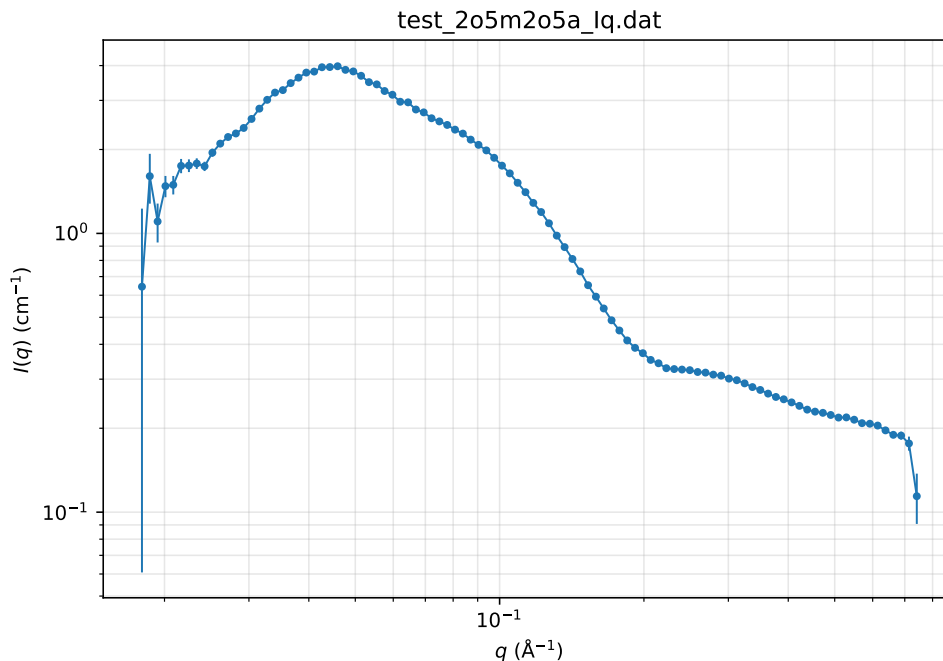


Figure 4. One-dimensional reduced data. A three-column $I(q)$ file plotted on logarithmic axes with error bars. Axis labels and units are applied automatically. This figure was produced by the same code path the **p** key invokes.

Figure 5 shows a real temperature series in the tiled view. It contains eight reduced $I(q_x, q_y)$ files from one sample measured from 20 to 55 °C. The whole series is selected in the file pane with one glob: `+,` followed by `*C_conf1_Iqxqy.dat`. After **p** is pressed, the files are placed in natural order, four panels per row, with one logarithmic color scale. The temperature-dependent change in the scattering ring can then be compared across the panels.

6.4 DETECTOR HEAT MAPS FROM NEXUS

Raw EQ-SANS (Zhao, Gao, and Liu 2010) event files are rendered as a 256×192 detector image, as shown in Figure 6. Processed Mantid files—including wavelength-banded `drt sans` output and event-workspace masks—render through the same view.

The image is reconstructed from the event arrays without starting Mantid. Therefore, the detector view is available on hosts without a Mantid environment and while a Mantid installation is being upgraded.

6.5 GENERIC TABULAR PLOTS

l plots whitespace-, comma-, or tab-delimited tables on linear axes and uses the header row for axis labels. This operation can be used with the batch metadata extractor in Section 7. For example, the user can extract temperature against time across a run series and plot the resulting table in the same program.

Enter opens static images, including PNG, JPEG, and TIFF files, in a matplotlib window. This can be used for figures written by autoreduction.

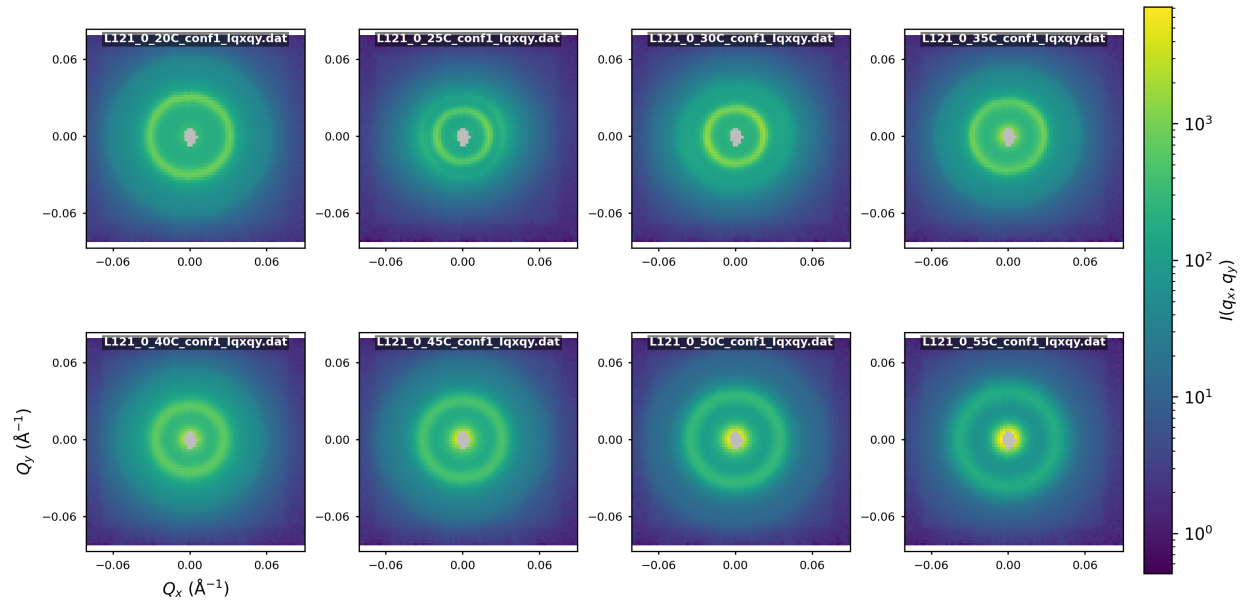


Figure 5. Tiled two-dimensional plot of a temperature series. Eight reduced $I(q_x, q_y)$ files (IPTS-36552, 20–55 °C) tagged with one glob and plotted with **p**. The shared logarithmic color bar makes the panels directly comparable; the isotropic scattering ring contracts and the low- q intensity grows as the sample is heated.

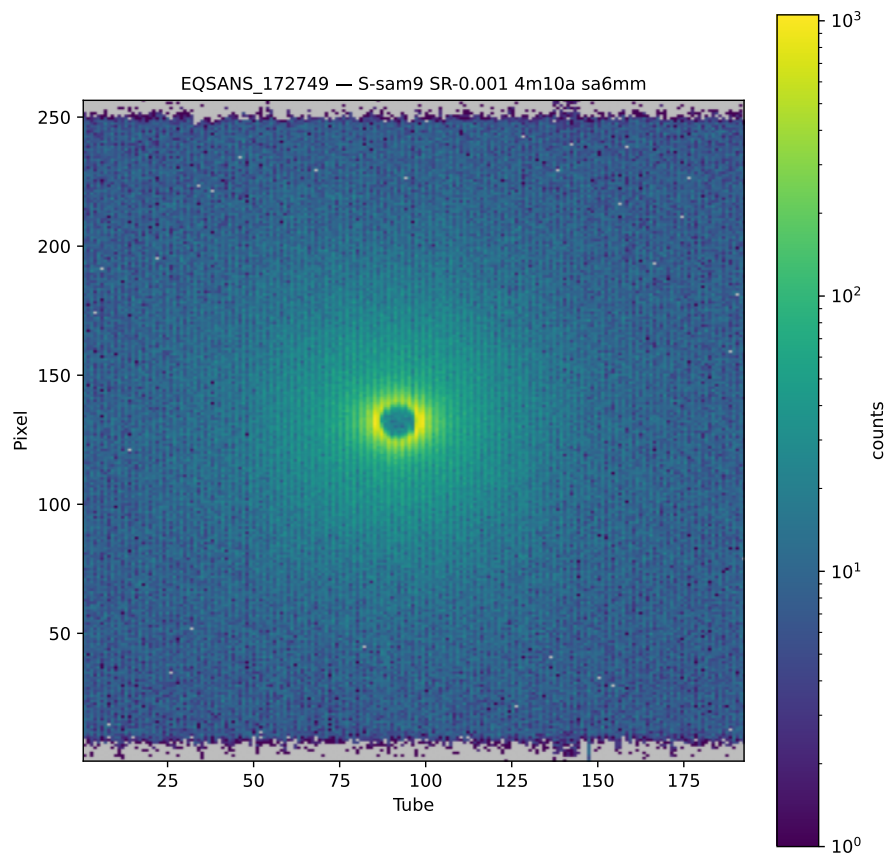


Figure 6. Detector heat map from a raw NeXus event file. Counts summed over all events and reordered into the physical tube-and-pixel layout, on a logarithmic color scale. The beam stop is visible at the center of the direct beam. The reconstruction is performed entirely in `numpy`; no Mantid runtime is involved.

7. METADATA INSPECTION AND EXTRACTION

7.1 SINGLE-FILE TREE BROWSER

m opens the file under the cursor in a tree browser. The hierarchy is loaded as needed, and only the direct children of an expanded group are read. A complete walk of a raw file that is several hundred megabytes can produce tens of thousands of entries and take several seconds. Loading only the opened groups avoids this delay and keeps the tree readable.

Selecting a leaf shows its full path, data type, shape, units from the `units` attribute, and a one-line preview of its value.

7.2 KEYWORD SEARCH

Loading only expanded groups reduces the file-reading cost, but the user still needs to find a path. Sample-environment logs can be stored at paths such as `/entry/DASlogs/BL6:SE:QNTC1:ReadProbeTemperature/value`, and reaching one by expanding groups requires knowing the instrument's device naming scheme.

Therefore, `/` in the tree browser opens a keyword search. A worker thread walks the full key list once and caches it. Later queries use a case-insensitive substring filter on the cached list. The results table shows matching paths and their shapes. The detail pane follows the highlighted row, so **Up** and **Down** can examine candidates while the search box remains open. Figure 7 shows a search for `temperature` in a real EQ-SANS file. It finds 28 matches from three devices without requiring knowledge of their complete paths.

Two limits keep the search responsive for large files. At most 500 matches are rendered. If more matches exist, the hint line reports that the results were truncated. Value previews also take a slice before an array is materialized. Thus, a preview of an event array with millions of elements reads five values instead of the full array.

7.3 BATCH EXTRACTION

M extracts selected keys from several files. It first opens a full-screen key picker. **Space** selects a key, and `/` searches all keys in the file with the same method used by the single-file browser. **Ctrl+S** returns to the output form.

Extraction uses a thread pool with eight workers by default, allowing several files to be read concurrently. The worker count can be changed for the available filesystem and host.

The key picker accepts either a dataset or a group containing a `value` child, as used by the SNS DAS log convention. Selecting such a group is the same as selecting `<group>/value`.

7.4 OUTPUT MODES

Two output modes support different questions.

Summary mode produces one row for each input file and reduces time-series values to their mean. It can answer, for example, what the temperature was for each of 40 runs. Optional `_stdev` and `_n` columns give the spread and sample count for each mean.

Per-file mode produces one table per input file and keeps the full arrays. It can show, for example, how temperature changed during one run. The placeholder `<filename>` in the output template selects this

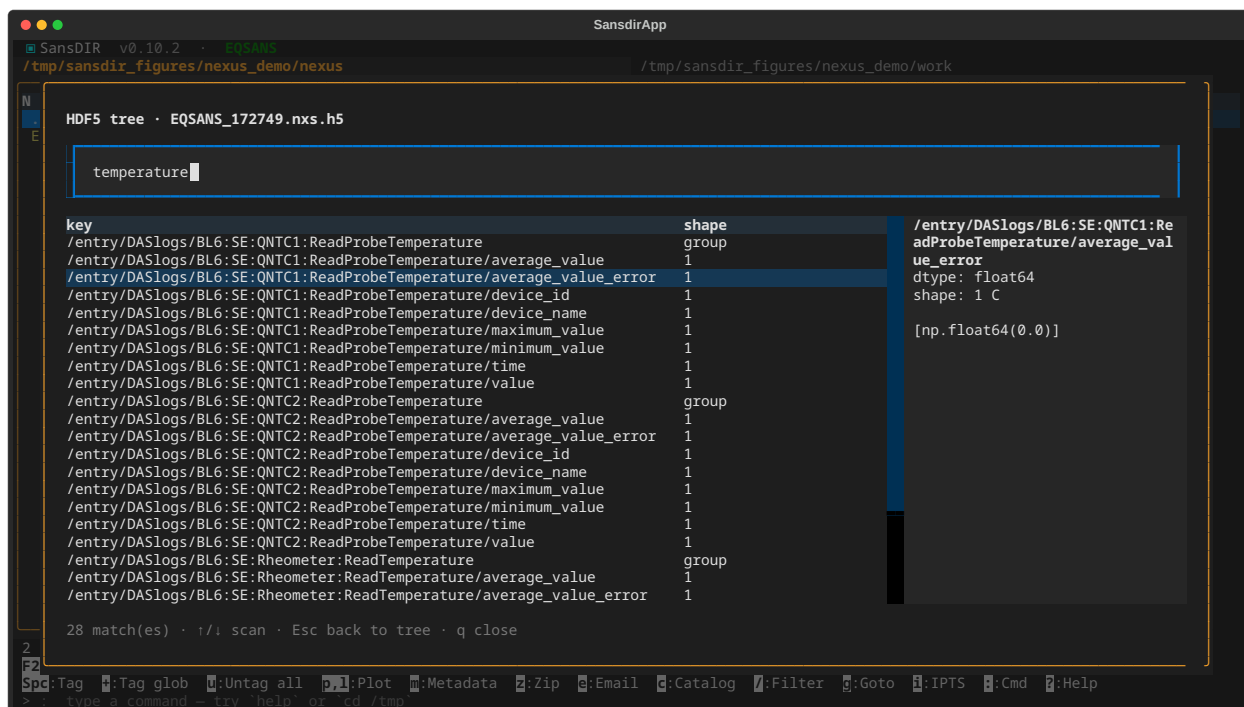


Figure 7. Keyword search in the HDF5 tree browser. A substring query against a raw EQ-SANS file. The left table lists matching keys with their shapes; the right pane shows the highlighted key’s dtype, shape, and value preview. **Esc** returns to the tree view, and a second **Esc** closes the browser.

mode, as in <filename>_temp.csv. If per-file mode is requested without the placeholder, the program adds <filename>_ to avoid repeatedly overwriting one file.

By default, output is written to the working directory of the inactive pane. The active pane often contains a read-only proposal directory, where writing the output would cause a permission error.

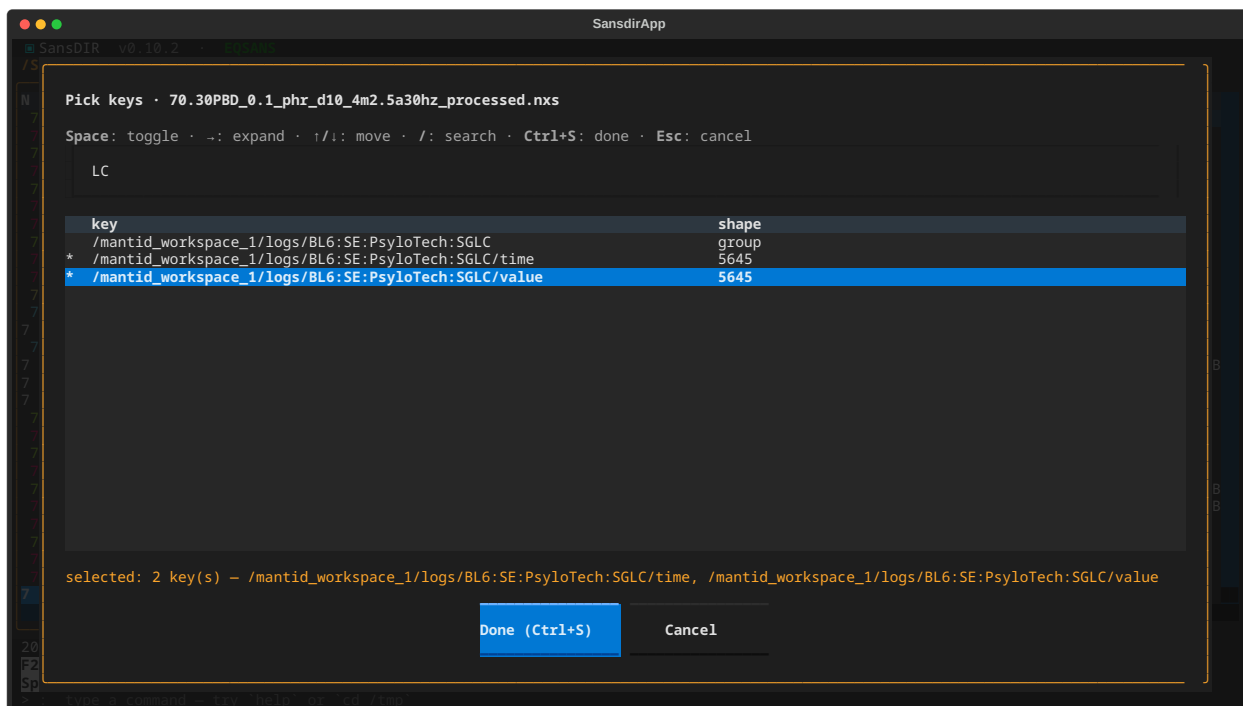
7.5 A WORKED EXAMPLE

Figures 8 and 9 show a complete extraction and plotting example using a real experiment file. The file is a Mantid-processed workspace from an EQ-SANS measurement with an in-situ mechanical load frame. The required quantity is the load-cell record, stored in the DAS log BL6:SE:PsyloTech:SGLC. The goal is to extract this record as a time series that can be plotted.

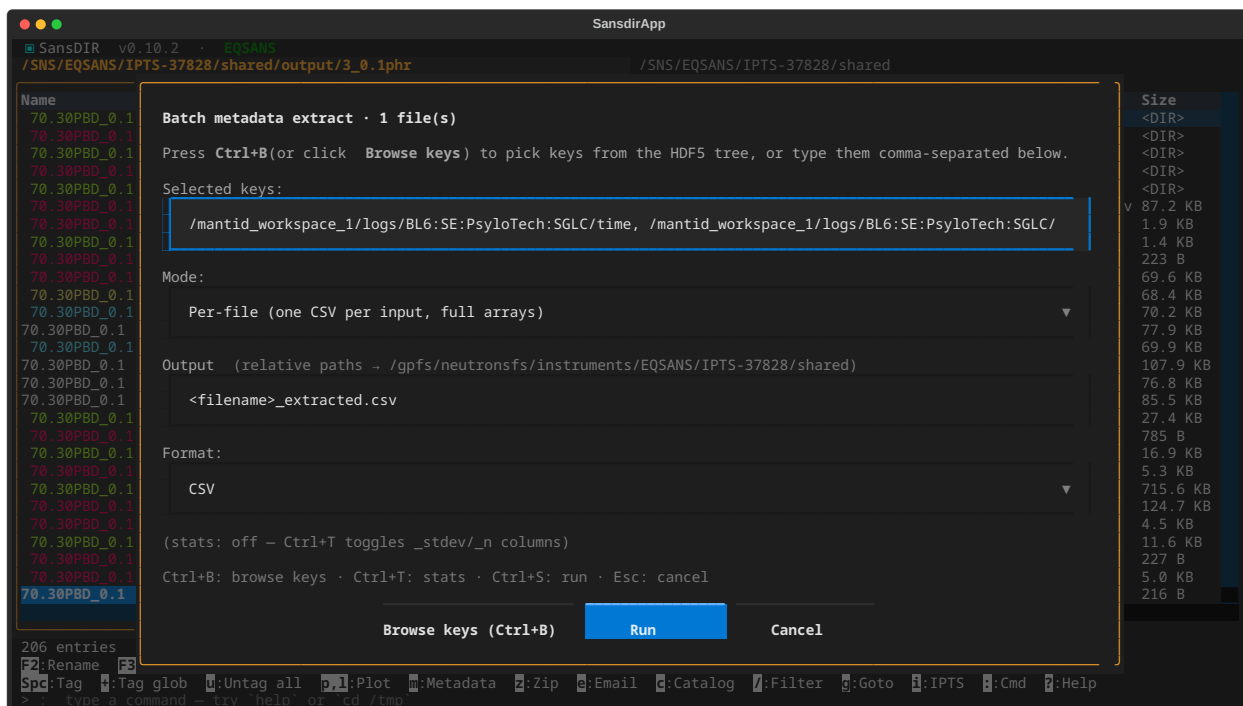
1. In the active pane, navigate to the file (IPTS-37828/shared/output/.../*_processed.nxs) and point the other pane at the directory that should receive the output, here the proposal’s writable shared/ folder.
2. Press **M** to open the key picker on the file’s HDF5 tree. Because the full DAS log path is not known, press **/** and enter the fragment LC. Three rows match. Press **Space** on .../SGLC/time and .../SGLC/value. Both rows are marked, and the status line shows *selected: 2 key(s)* (panel a).
3. Press **Ctrl+S**. The picker folds back into the output form with both keys filled in. Per-file mode and the <filename>_extracted template are the defaults. Change the format to comma-separated values (CSV) and press **Ctrl+S** again to run the extraction (panel b).
4. The extractor writes <filename>_extracted.csv in the inactive pane’s directory. The file has two columns, time and value, and 5 644 rows. The pane refreshes and the new file appears (panel c).

5. Press **Tab** to switch panes, put the cursor on the new CSV, and press **1**. The generic linear plotter reads the header row for axis labels and draws value against time (panel d). The load increases during the first 450 seconds of the run and then releases sharply. Without this workflow, the file would need to be exported and opened in a separate plotting tool for the same check.

The four panels were captured from the running program and show its actual fonts, colors, and layout.

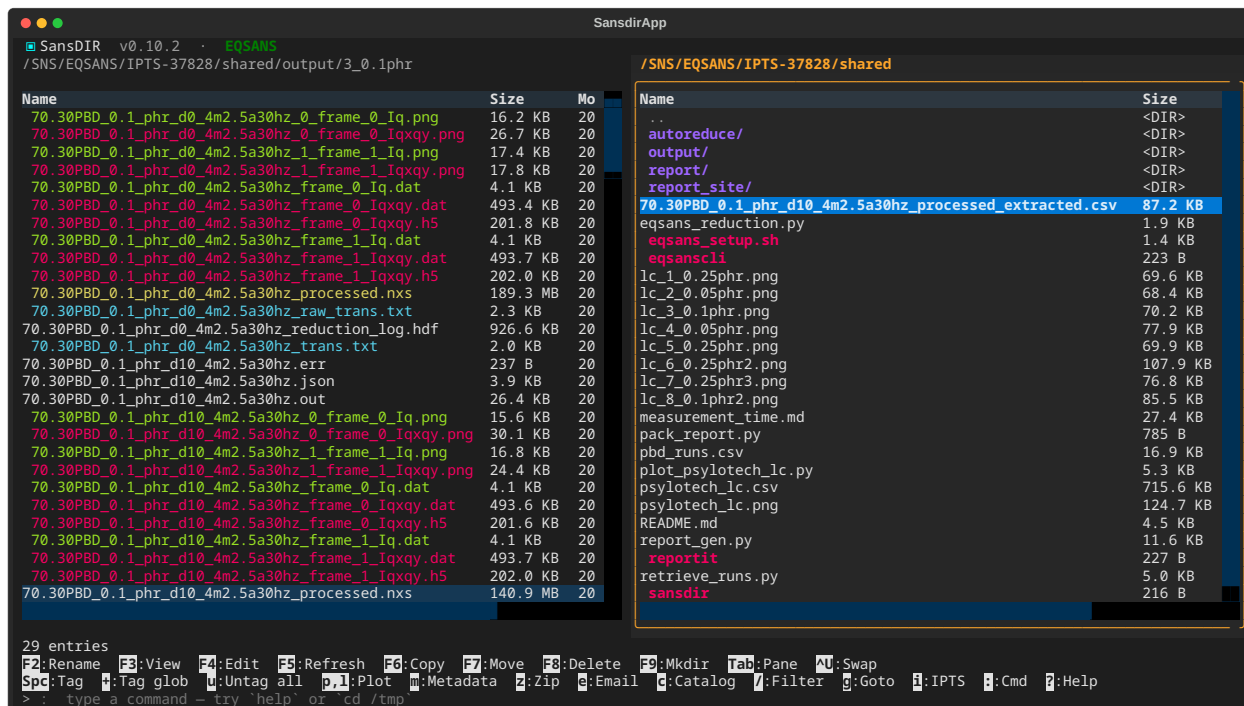


(a) **M**, then **/ LC**: search and select keys

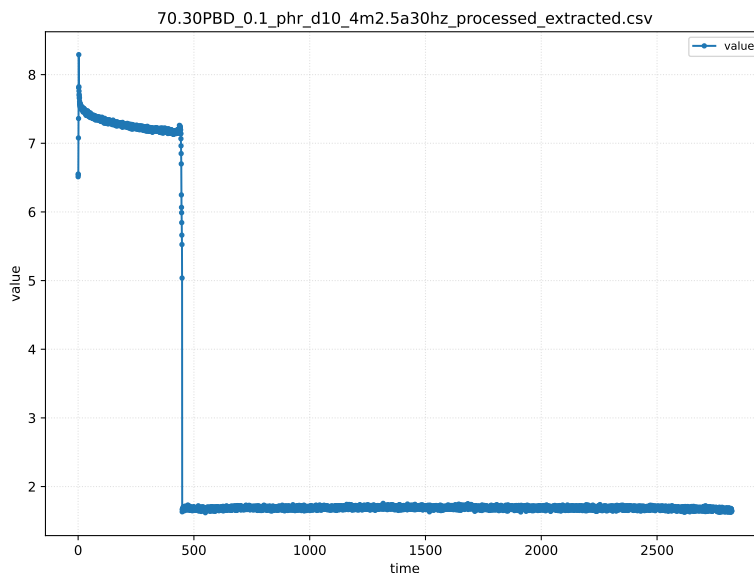


(b) **Ctrl+S**: the output form, format set to CSV

Figure 8. Batch metadata extraction (1 of 2). (a) The key picker after pressing **M** and searching for LC: both datasets under the PsyloTech:SGLC log are selected. (b) The output form with the chosen keys, per-file mode, and CSV format.



(c) the extracted CSV lands in the other pane



(d) 1: plot of the extracted table

Figure 9. Batch metadata extraction (2 of 2). (c) The destination pane after the run: the extracted CSV is under the cursor. (d) The curve produced by pressing 1 on it—the load-cell trace over the course of the run.

8. CATALOG INTEGRATION

8.1 AUTHENTICATION

OnCat ([Oak Ridge National Laboratory 2025b](#)) requires an Open Authorization version 2 (OAuth2) bearer token for read-only endpoints. The client uses the client-credentials grant to obtain a token and caches it until shortly before it expires. Credentials are read from the `[oncat]` configuration section or the `ONCAT_CLIENT_ID` and `ONCAT_CLIENT_SECRET` environment variables. Working defaults allow a cluster user to search without additional configuration. The endpoint is also configurable for use with a staging instance.

8.2 SEARCH MODEL AND CACHING

OnCat does not provide a fuzzy keyword endpoint. Therefore, the client requests all experiments for the configured instrument, stores the list on disk, and filters it locally. The filter uses substring matches against the proposal identifier, title, and member list. The configurable cache time to live (TTL) is 24 hours by default. The first search requires one server request, while later searches use the cache and complete in well under a second.

A stale cache may not contain a newly allocated proposal. The **r** key in the browser therefore requests a new list without using the cache.

Two measures keep the browser responsive for large catalogs. A 200 ms quiet period is applied to the filter input, so the result list is not rebuilt after every keystroke during fast typing. The browser also renders at most 200 rows and reports how many additional matches exist.

8.3 RUN CATALOG PANE

Selecting a proposal opens its `shared/` directory in the active pane and loads its run list in the right pane. **c** shows or hides the catalog. In the catalog, **Space** tags runs and the same keys used in the file panes act on the selection. **Enter** or **p** plots the raw detector image, **m** opens the HDF5 tree, **M** extracts metadata, and **K** opens the mask editor. Thus, the catalog provides another selection view with the same commands as the file panes.

9. DETECTOR MASK CREATION

9.1 ARCHITECTURAL CONSTRAINTS

Detector masking produces an input to reduction instead of displaying reduction output. Its implementation follows three constraints:

- **No Mantid import in the main process.** Mask drawing and array creation use Python, numpy, and h5py. At save time, a standalone driver imports Mantid inside an external `drtsans` subprocess. The TUI process does not import Mantid.
- **EQ-SANS geometry is isolated.** Detector shape, tube reordering, and the 48-bank mapping are kept in the detector and bank/tube modules. When the source NeXus file provides a pixel-identifier array, the program uses it to connect the displayed pixels to detector identifiers.
- **Pixel ordering is defined in exactly one place.** One module establishes that the flattened mask array and pixel identifier array are aligned element by element. The output writers use this ordering instead of calculating it again.

The mask uses the Mantid convention: 1 is a masked pixel and 0 is a retained pixel. Therefore, `MaskDetectors` excludes the cells selected by the user. One flag inverts the final mask.

9.2 INTERACTIVE EDITOR

From a file pane or the run catalog, **K** opens a matplotlib window containing the detector heat map of the selected file. The user draws mask shapes on this image.

The cell aspect ratio is 1/1.3 to account for the EQ-SANS detector tube pitch of about 5.2 mm and pixel pitch of about 3.9 mm. Without this correction, an ellipse that appears circular on the screen would not be circular on the detector. The canvas also extends several cells beyond the detector boundary, which is marked by a faint dotted line. A selection can then begin outside the active area. Otherwise, masking the outermost row would require the first click to be exactly on pixel zero.

Table 9 lists the editor's controls.

Table 9. Interactive mask editor controls

Key	Action
r	Rectangle draw mode
e	Ellipse draw mode
v	Edit mode: click to select a shape, drag to move it, Delete to remove it
k	Open the bank and tube specification dialog
z	Undo
i	Invert the mask
s	Save, through a file chooser
Esc	Quit without saving

The bank and tube dialog accepts a compact notation. For example, `b3` selects a complete bank, `t50` selects a tube, and `b5-7 t10-15` selects mixed ranges. This is useful for strip masks that are difficult to draw by hand. The resulting rectangles are ordinary shapes and are stored in the mask log like other shapes.

Two changes improved the response of the editor. When a shape is moved, the static canvas is saved at mouse-down and only the moving area is drawn for each motion event. The full heat map is not redrawn. Also, bank and tube input uses a dialog instead of an inline matplotlib text widget. The inline widget redraws after every keystroke, which produces visible input delay on a logarithmically scaled 256×192 image.

As the pointer moves, a cursor readout shows tube, pixel, counts, bank, and the tube index within its bank. These values help the user convert a feature in the image into a bank and tube specification.

9.3 COMMAND-LINE MASK CREATION

The same operations are available from the command line. Shapes are specified in pixel coordinates, and each option can be repeated:

```
sansdir mask /SNS/EQSANS/IPTS-XXXXX/nexus/EQSANS_172749.nxs.h5 \  
--circle 96,128,12 \  
--rect 0,0,15,15 --rect 176,0,191,15 \  
--rect 0,240,15,255 --rect 176,240,191,255 \  
--output beam_stop.nxs
```

The available shapes are `-rect X0,Y0,X1,Y1`, `-ellipse XC,YC,RX,RY`, `-circle XC,YC,R`, and `-polygon X1,Y1,X2,Y2,...` with at least three vertices. Each output has a sidecar JavaScript Object Notation (JSON) log. The `-shapes-json` option reads this log, allowing a mask created interactively to be applied to other runs in a script.

9.4 OUTPUT FORMATS AND REDUCTION COMPATIBILITY

The program supports three output formats: a Mantid SaveMask Extensible Markup Language (XML) file, a NeXus file, and a `numpy` array with a metadata sidecar.

The NeXus output requires an additional step. A mask used by the `drt sans` reduction pipeline must contain per-detector mask flags in the instrument parameter map read by the pipeline. Mantid is used to write this structure. During saving, the program calls the cluster's `drt sans` wrapper and runs a load-mask-save sequence on the source file. If Mantid is not available, a pure-`numpy` writer produces a file for visualization only. The TUI briefly identifies this as a legacy save, while the CLI does not give a prominent fallback warning. This file can be inspected but does not change a reduction.

The visualization file follows the convention of existing EQ-SANS beam-stop files. Each *unmasked* detector contains one synthetic event, while masked detectors contain no events. Therefore, a mask opened with **p** shows the masked area in grey, like a beam stop. The mask builder continues to use Mantid's "1 means masked" convention internally; only the file encoding is inverted.

10. NON-INTERACTIVE COMMAND-LINE INTERFACE

Several operations are available without the TUI for scripts and batch jobs. The Click CLI is a separate entry path, but it calls the same service functions used by the interactive commands.

10.1 LAUNCHING THE INTERFACE

```
sansdir                                # TUI in the current directory
sansdir /SNS/EQSANS/IPTS-12345/shared  # TUI rooted at a directory
```

A path given without a subcommand is interpreted as `sansdir tui PATH`. Thus, the subcommand name can be omitted for normal interactive use.

10.2 BATCH METADATA EXTRACTION

```
# Summary table: one row per file, time series reduced to means.
sansdir extract \
  -k /entry/DASlogs/temperature/value \
  -k /entry/duration \
  --out summary.tsv \
  /SNS/EQSANS/IPTS-12345/nexus/EQSANS_*.nxs.h5

# Per-file tables: each input gets its own CSV with full arrays.
sansdir extract \
  -k /entry/DASlogs/temperature/time \
  -k /entry/DASlogs/temperature/value \
  --out '<filename>_temp.csv' \
  EQSANS_172749.nxs.h5 EQSANS_172750.nxs.h5

# Add spread and sample-count columns (summary mode only).
sansdir extract --with-stats -k /entry/DASlogs/temperature/value *.nxs.h5
```

The options are `-k/-keys` (repeatable and required), `-o/-out`, `-format` with the values `tsv`, `csv`, or `columns`, `-with-stats`, and `-workers` for the thread-pool size. Each written path is printed on a separate line, allowing the command to be used in a shell pipeline.

10.3 MASK CREATION

Section 9 describes the `mask` subcommand. Its options are `-rect`, `-ellipse`, `-circle`, `-polygon`, `-shapes-json`, `-inverse`, `-output`, and `-format` with values `nxs`, `xml`, or `npz`. When complete, it reports the output path, masked pixel count as a number and percentage, and sidecar log path.

11. CONFIGURATION

Configuration is read from `~/.config/sansdir/config.toml` or from the path given by `$SANSDIR_CONFIG`. All sections and keys are optional, and the program can run without a configuration file.

```
[ui]
theme = "monokai"      # any Textual theme name
colorbar_mode = "shared"  # tiled 2-D scale: "shared" or "independent"

[keys]
"ctrl+y" = "view.toggle_hidden"  # bind a key to a compatible command
f5 = "ui.move_tagged"            # replace the default F5 action

[oncat]
default_instrument = "EQSANS"
cache_ttl_seconds = 86400

[instrument]
default = "EQSANS"  # "" falls back to [oncat].default_instrument
auto_detect = true  # a /SNS/USANS/... launch path overrides 'default'

[usans]
pixi_manifest = "/usr/local/pixi/usansred"
reduce_command = ""  # explicit override for the engine argv
data_dir_template = "/SNS/USANS/{ipts}/shared/autoreduce"
logbin = true
thickness_cm = 0.1
short_name_copy = true  # also write UN*_bsub.txt aliases

[mail]
command = "mail"  # or "mutt"
default_subject = "[sansdir] data"
```

The loader handles several invalid settings without stopping the program. An unknown theme name is replaced by the default and reported to the user. A key binding that names an unknown command is removed and also reported. A malformed TOML file or an invalid value type can still stop startup and must be corrected in the configuration file.

A custom key binding is most suitable for a command that requires no arguments. The configuration override stores a key and command name but does not add the argument resolver used by some built-in bindings.

The theme can also be changed during a session with `:theme <name>`. Entering `:theme` without an argument lists the available themes.

12. USANS REDUCTION

USANS uses many operations that were already available in the program: the proposal catalog, file manager, one-dimensional plotting, and editor. It adds a short reduction workflow and a desmearing step. The instrument team's `usansred` package performs the reduction. The desmearing operation uses the truncated Abel inversion of Huang (Huang et al. 2026). These operations are included through an *instrument mode* and a small, self-contained module instead of a separate application.

12.1 INSTRUMENT MODE

The mode is either SANS or USANS. It is resolved once at launch, using path detection first, then `[instrument].default`, and finally `[oncat].default_instrument`. A launch path containing `usans` selects USANS. This includes `/SNS/USANS/IPTS-N` and a user's own scratch directories. Path detection has priority over the configuration file because the path gives the current working context. The command `instrument.set`, entered as `:instrument usans`, can change the mode during a session.

The mode controls four items: the instrument queried by the proposal browser, the columns shown in the run catalog, the action of **Enter** on a catalog row, and whether the **r** and **d** bindings are active. USANS has no moveable detector and uses a fixed incident wavelength, so the distance and wavelength columns would contain only constants and are omitted. The title bar shows the mode, and the reduction and desmearing actions appear in the key hint bar in USANS mode.

The mode changes *bindings*, not command registrations. The USANS commands are registered in both modes, so `:usans-reduce` works even if the user did not switch modes; only the key bindings are hidden. The live help overlay and hint bar request the key map for the current instrument mode. Calling `default_keymap()` without a mode still returns the original SANS map for backward compatibility.

12.2 THE SETUP TABLE

The reduction engine reads a small CSV table with one row per sample. Each row contains a flag that distinguishes the empty-cell background from the samples, a name used as the output filename stem, a first run number, a scan count, a thickness, and optional runs to exclude. The program generates a draft from the proposal run list. The user then reviews and corrects it with **F4**, using the same editor as for other files. A separate table editor is not needed.

During generation, consecutive runs with the same title are grouped into blocks. The block whose title identifies an empty cell or banjo is selected as the background. The scan count requires additional checking. A block of N consecutive runs contains $(N - 1)$ analyzer rocking-curve scans and one transmission run, while the engine reads only the rocking-curve files. The value of N depends on the experiment settings. Therefore, using the catalog run count would make the engine look for an input file that does not exist for the transmission run. Instead, the program counts the runs that have rocking-curve files on disk. This works for different block sizes without additional configuration. A gap inside a block is recorded in the exclude column of the CSV.

Some cases still require review. If a measurement was stopped and restarted under one title, it can produce an oversized block in which only the last few runs are usable. The generator keeps the last window of the modal size and flags the block. A companion notes file records every such choice: narrowed blocks, unusual sizes, blocks with no reducible runs, removed transmission runs, and the selected background. The generated table is a first draft. The notes provide the information needed to review it.

12.3 INVOKING THE ENGINE

The instrument team's `usansred` package contains the reduction mathematics and is deployed on the cluster as a `pixi` environment. The program locates its console script and runs it as a subprocess. It does not reimplement the physics, and no `usansred` code is copied into this code base. The caller also handles two deployment conditions.

First, the console script can inherit the user's `~/ .local` site packages. On the analysis nodes, an old `pandas` package in this location causes the script to stop. Therefore, each invocation disables user site packages and clears `PYTHONPATH`.

Second, the engine locates each run relative to the directory containing the setup CSV, not relative to the working directory. The reviewed table is normally in the user's folder, while the preprocessed data are in the proposal's autoreduce directory. The program must not write to this instrument data directory, which is also often read-only. It instead creates a temporary directory containing symbolic links to the data, copies the table there, and runs the engine in that directory. It does not modify anything below the instrument data root.

12.4 ONE KEY

One binding provides both operations. If the cursor is on a setup table, **r** reduces it. If the cursor is not on a table, the program offers to build one for the proposal identified by the current path. It fetches the run list, shows the catalog in the other pane, writes the table, and leaves the cursor on the new file.

The program stops after writing the draft and does not begin reduction. The user can review the table and the choices recorded in the notes before pressing **r** again. A table that can be parsed but fails validation is not regenerated. For example, it may contain two backgrounds or a zero scan count. In this case, the table may include real user corrections, so regeneration could discard them. The program reports the problem and directs the user to **F4**.

The reduced output is ordinary two- or three-column text and is handled by the existing plotting path. Only a filename rule was added to the classifier. The rule uses the filename rather than the column count, so a sample name containing `trans` is not incorrectly sent to the transmission plotter.

The engine gives the final curve the suffix `_background_subtracted.txt`. After each reduction, the program also writes a shorter `_bsub.txt` name. It keeps the original so that the engine's summary spreadsheet and other tools that expect the standard name continue to work. The shorter file is a copy, not a symbolic link, and is controlled by `[usans].short_name_copy`.

12.5 A WORKED EXAMPLE

Figures 10 and 11 show the complete workflow for a real proposal (IPTS-37679). It starts with an empty working folder and ends with reduced curves, using the **r** key and the related prompts.

1. Open the program in a writable working folder under the proposal path. The `/SNS/USANS` prefix selects USANS mode automatically.
2. Press **r**. The folder holds no setup table yet, so the program offers to build one for the IPTS named by the path (panel a). Confirm with **y**.
3. The run list is fetched from the catalog and displayed in the other pane, and a prompt asks for the first run of the experiment—here 49434 (panel b). Blocks recorded before that run, including beam-cycle alignment and test scans, are recorded in the notes but omitted from the table.

4. The program writes `IPTS-37679_setup.csv` and its companion `NOTE.md`, leaves the cursor on the table, and previews its content in the other pane (panel c). In this case, it contains one background and 23 sample blocks. The user checks the selected background and block sizes against the NOTE and edits the table with **F4** if needed.
5. Press **r** again, now on the table. The data directory defaults to the proposal's `autoreduce` folder and the output directory to `output/` beside the table; accepting both runs the engine over every block. This takes seconds because it reads the preprocessed per-run ASCII files. On completion, the program offers to show the output folder in the other pane (panel d), where each sample now has its `UN_*_det_1_*` curve family. **p** plots any of them, and **d** desmeares (Section 12.6).

12.6 DESMEARING

USANS instruments use slit collimation instead of pinhole collimation. The larger illuminated area provides the flux needed for the high angular resolution of the Bonse–Hart crystal optics. However, the intensity recorded at a nominal momentum transfer Q is an average of the true intensity over the vertical acceptance,

$$I_{\text{exp}}(Q) = \frac{1}{\sigma_y} \int_0^{\sigma_y} I(\sqrt{Q^2 + s^2}) ds, \quad (1)$$

where σ_y is the vertical resolution half-width. For the SNS instrument, $\sigma_y \approx 0.13 \text{ \AA}^{-1}$, which is two decades above the top of the measured window. Desmearing recovers $I(Q)$ from $I_{\text{exp}}(Q)$. This is an ill-posed problem because the slit average couples each measured point to scattering at much higher Q .

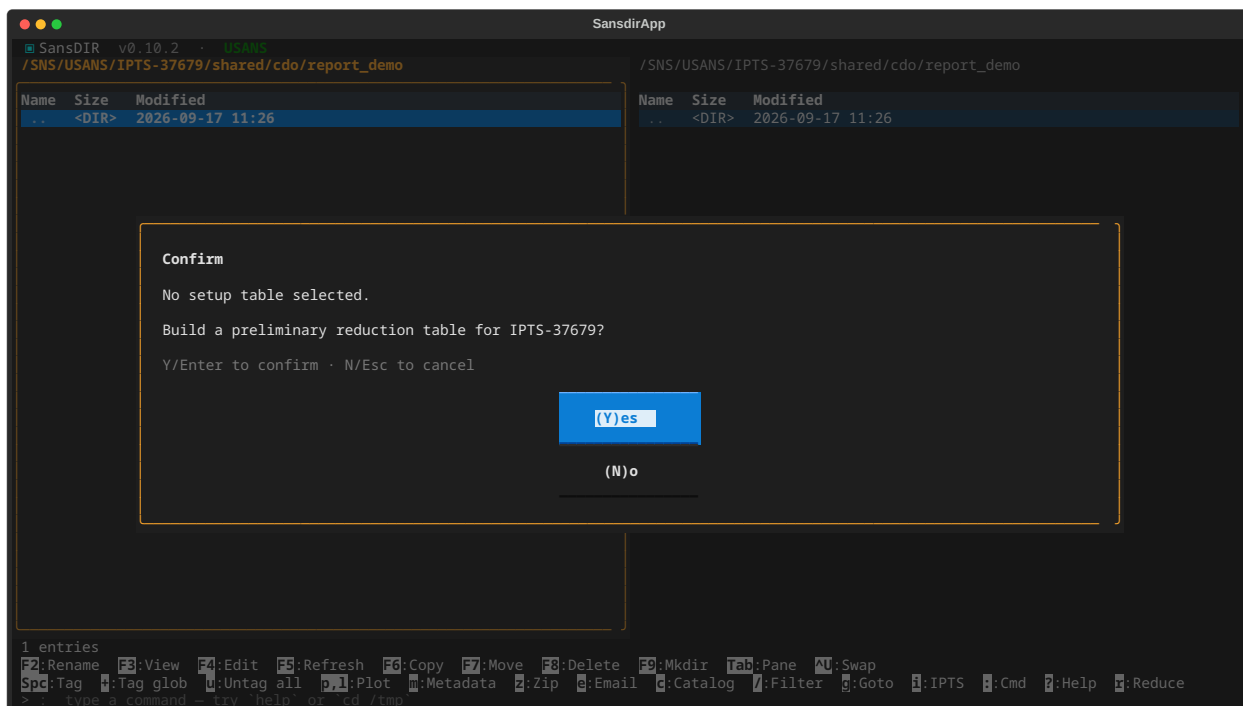
The **d** key desmeares the selected reduced curves. The program uses the non-iterative truncated Abel inversion of Huang (Huang et al. 2026). The method expresses the intrinsic intensity in closed form as

$$I(Q) = I(Q_m) - \frac{2\sigma_y}{\pi} \int_Q^{Q_m} \frac{I'_{\text{exp}}(Q_x)}{\sqrt{Q_x^2 - Q^2}} dQ_x, \quad Q_m = \sqrt{Q^2 + \sigma_y^2}. \quad (2)$$

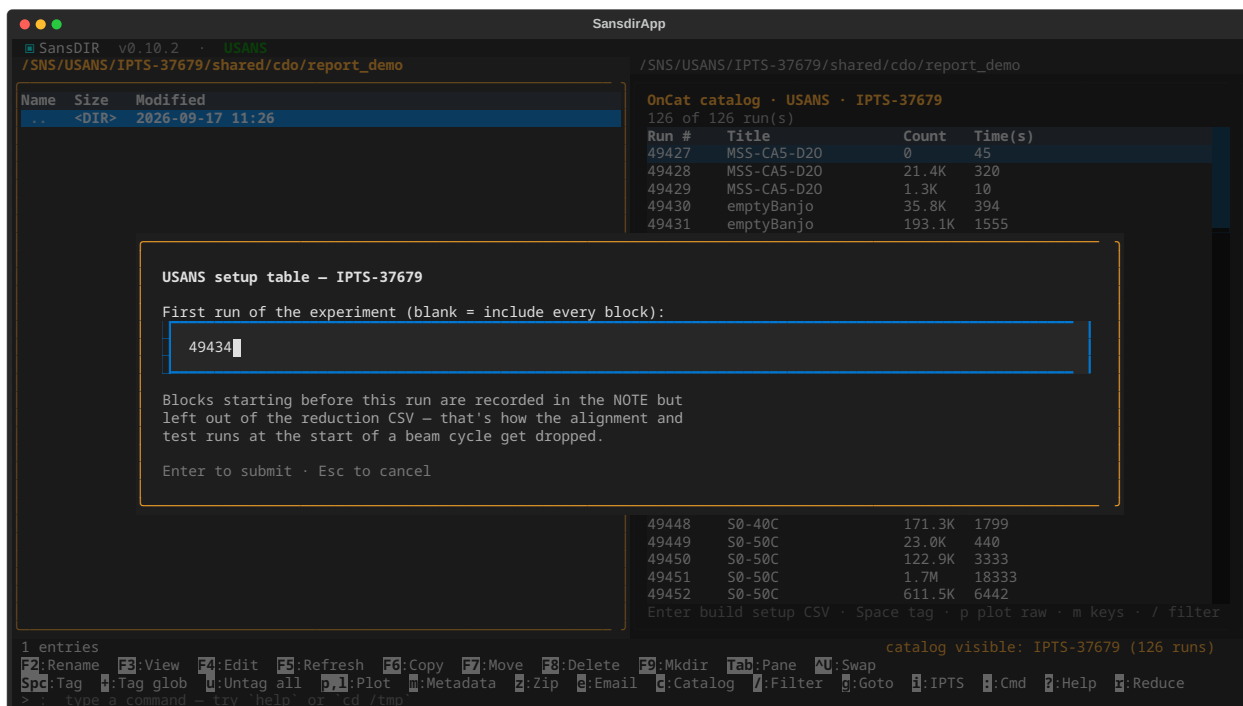
Because the expression is closed-form, the program propagates uncertainty through one linear operator instead of across iterative updates. This differs from the widely used iterative Lake method (Lake 1967). Each integral is evaluated with the substitution $Q_x = \sqrt{Q^2 + s^2}$, which removes the $1/\sqrt{Q_x^2 - Q^2}$ singularity at the lower limit. A Bayesian Gaussian-process alternative has also been reported for this inversion (Tung et al. 2026). The program uses the Abel method because it is deterministic and provides closed-form error propagation.

Equation (2) evaluates I at $Q_m \approx \sigma_y$ and integrates to that point. Therefore, it requires scattering data well beyond the USANS window. This requirement produces one interactive choice. When one curve is selected, the program asks whether a companion pinhole SANS measurement of the same sample is available. The question appears in the current pane, while the other pane remains a filterable file list. If a companion curve is selected, the two curves are placed on a common smearing basis, scale-matched, joined across the instrument gap, and inverted over the full range. If the user declines, or if several curves are desmeared together, the high- Q side is extrapolated as a power law to σ_y . This is the approach used by the National Institute of Standards and Technology (NIST) IGOR/IRENA package (Kline 2006). The result is then limited to the measured USANS range, and the extrapolation is recorded in the output header. Huang (Huang et al. 2026) show that a reconstruction without measured high- Q data is reliable only at low Q . The limited output range and header note make this restriction visible.

The desmeared curve is written as a three-column Q , I , dI text file, which the **p** key plots in the same way as other curves. The inversion uses only `numpy` and adds no dependency. It was validated by a

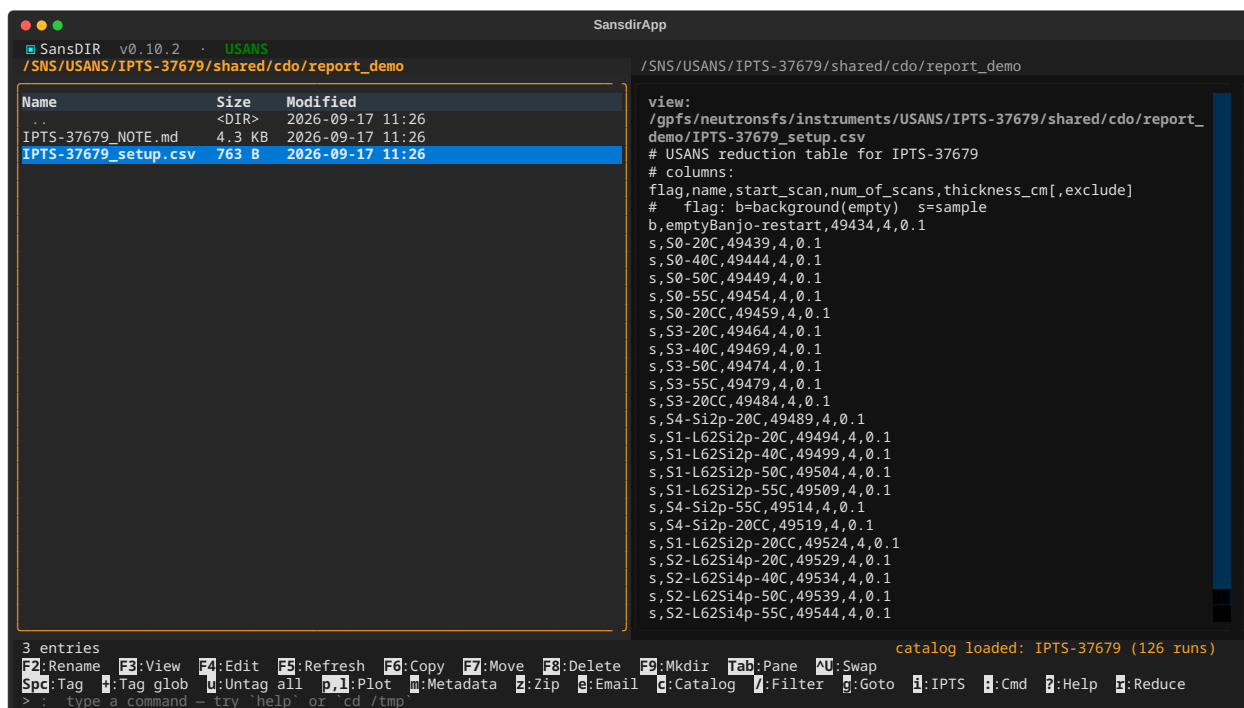


(a) **r** in an empty working folder: offer to build the setup table

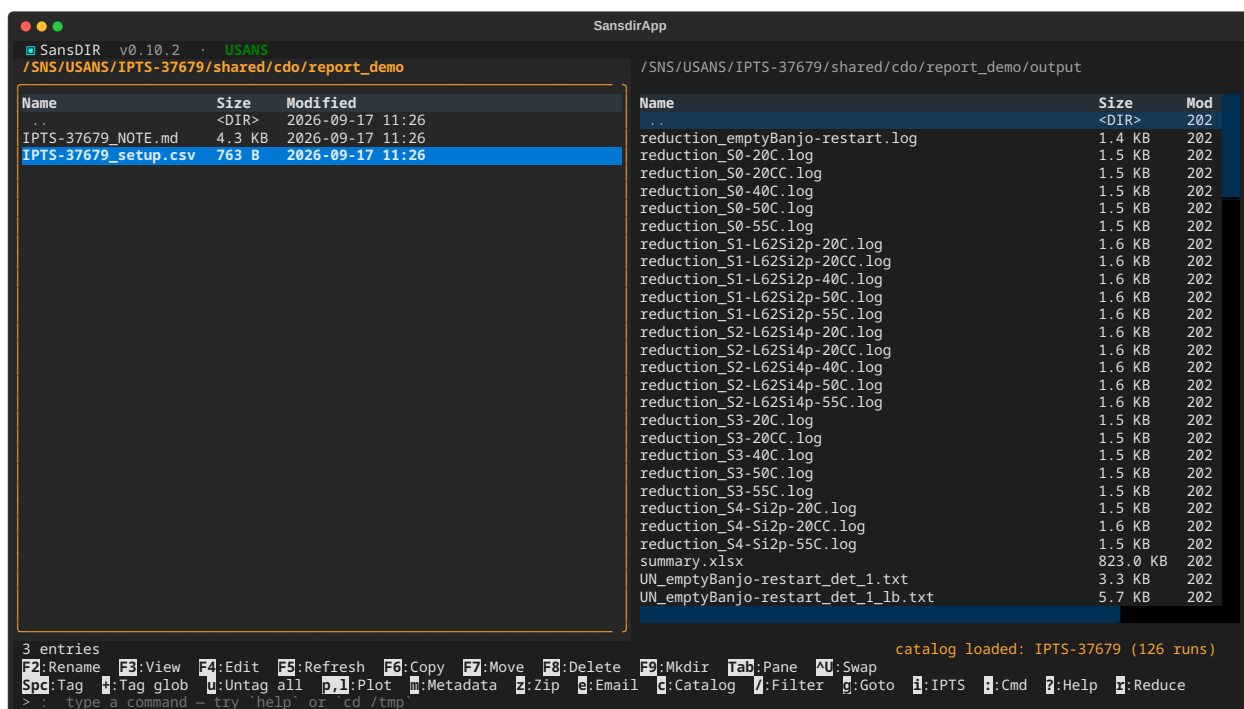


(b) the start-of-experiment prompt: run 49434

Figure 10. USANS reduction, end to end, on one key (1 of 2). (a) **r** in an empty working folder offers to build the setup table for the IPTS named by the path. (b) The start-of-experiment prompt (run 49434); the fetched run list is visible in the other pane.



(c) the generated setup CSV, previewed in the other pane



(d) after the second **r**: the reduced curves in the other pane

Figure 11. USANS reduction, end to end, on one key (2 of 2). (c) The generated table under the cursor, previewed in the other pane for review. (d) After the second **r**: the reduced curves in the output folder.

smear–desmear round trip using analytic power laws. The tests require the recovered log–log slope to agree within 0.03 and the median core intensity error to remain below two percent. The reported uncertainty increases near the top of the range, where the high- Q extrapolation is less reliable.

13. SUPPORTED DATA FORMATS

13.1 ASCII REDUCED DATA

Reduced data files are whitespace-delimited ASCII. Optional comment lines begin with #. Table 10 lists the recognized column layouts. The program counts the columns in the first non-comment line.

Table 10. Recognized ASCII column layouts

Kind	Columns	Layout	Treatment
$I(q)$	2	$q, I(q)$	Logarithmic axes
	3	$q, I(q), \sigma_I$	Logarithmic axes with error bars
	4	$q, I(q), \sigma_I, \sigma_q$	As three-column; the resolution column is not plotted
Transmission	2	$\lambda, T(\lambda)$	Linear axes, λ in Å
	3	$\lambda, T(\lambda), \sigma_T$	With error bars
$I(q_x, q_y)$	4	q_x, q_y, I, σ_I	Heat map on the recovered q grid
	6	$q_x, q_y, I, \sigma_I, dq_x, dq_y$	As four-column

Note: Classification does not rely on column count alone; the filename and any header keywords are considered first.

13.2 NEXUS FILES

Raw event-mode files follow the SNS NeXus conventions ([NeXus International Advisory Committee 2025](#)). The program uses the paths `/entry/bank<N>_events/event_id` for detector events, `/entry/instrument/bank1/pixel_id` when an explicit pixel-identifier array is available, `/entry/DASlogs/<name>/value` for sample environment values, and `/entry/DASlogs/<name>/time` for the corresponding time axis. Scalars such as `/entry/duration` are read directly.

Processed Mantid files are also accepted for detector viewing and metadata extraction. Mantid writes these files with a `.nxs` extension but without the `.h5` suffix. Therefore, classification checks the HDF5 magic signature instead of using only the filename.

14. TESTING AND QUALITY ASSURANCE

14.1 TEST SUITE

For version 0.10.2, pytest collects 805 test cases from 51 `test_*.py` modules. Six test functions are interface snapshot comparisons. Integration tests that require a real NeXus file or the installed reduction engine are skipped when those resources are not available. The exact pass, skip, and run-time results depend on the test environment and are not used as fixed values in this report.

The tests follow the software layers:

- **Unit tests** cover every module under `core/`, `hdf/`, `mask/`, and the plot classification logic.
- **Registry tests** verify duplicate-name rejection, parameter validation, argument checking during dispatch, and that every key binding names an existing command.
- **Interface tests** drive the real application through Textual’s pilot harness, pressing keys and asserting on resulting state. The feature of Section 7.2, for example, is covered by tests that press `/`, type a query, wait for the worker, and assert on the resulting match list, the case-insensitivity of the match, the two-stage **Esc** behavior, and the empty-result path.
- **Snapshot tests** compare rendered output against stored references to catch unintended layout regressions.

Two restrictions apply to the tests. OnCat tests use mock representational state transfer (REST) responses and do not contact the real service. Tests also do not access real cluster data paths. Their fixtures are generated or taken from a small set of committed files.

14.2 STATIC ANALYSIS AND STYLE

Linting and formatting use `ruff` with a line length of 100 and the `E`, `F`, `W`, `I`, `B`, `UP`, `N`, `C4`, `SIM`, and `RUF` rule sets enabled. Type checking uses `mypy` in strict mode over `src/sansdir`. Public functions have type hints and Google-style docstrings. Bare `except` clauses are not allowed. Exceptions are caught specifically, logged, and shown to the user in the status bar.

CI runs the suite on Python 3.10, 3.11, and 3.12 on Linux.

15. PERFORMANCE

Table 11 lists the performance budgets. Startup is measured. The other values are targets used to identify regressions. When a target is missed, the operation is profiled with `pyinstrument`.

Table 11. Performance measurement and regression budgets

Operation	Value	Basis
<code>sansdir -help</code> startup	~120 ms median ^a	Measured
Render a directory of 1,000 entries	100 ms	Regression budget
OnCat search, first call	2 s	Regression budget
OnCat search, cached	100 ms	Regression budget
Single metadata key read	50 ms	Regression budget
Batch extract, 100 files, 3 keys	10 s	Regression budget

^a Median end-to-end wall-clock time for `sansdir -help` on an analysis node, re-measured after the USANS reduction and desmearing features were added; the best case is near 60 ms. The figure did not grow with those features because the fast path imports none of the scientific stack (Section 3.6).

Three measures help keep these operations within their budgets. Directory listings use `os.scandir` and reuse each entry's stat result during one scan. The OnCat browser debounces its filter input and limits the number of rendered rows. HDF5 value previews take a slice before materializing an array, so the preview cost does not depend on the full length of an event array.

16. SUMMARY

sansdir puts the file handling, inspection, and packaging tasks around SANS reduction in one keyboard-driven terminal application. It also keeps USANS setup, reduction, and desmearing in the same workflow. It does not require a display server and runs from a shared installation on the ORNL analysis cluster without per-user setup. The measured `sansdir -help` startup has a median of about 120 ms; the TUI first-paint time is not reported here.

Every interactive action is a typed command in one registry. Key bindings and the interactive command line use this registry. The non-interactive Click CLI uses a separate path to the same service functions. The command data also generate the interactive help, while the service logic can be tested without the terminal framework.

The present version has two main limitations. The detector heat map and mask editor assume the EQ-SANS detector layout and have been tested only with EQ-SANS data. The navigation and file-format operations are not specific to one instrument, but they have not been verified on the High Flux Isotope Reactor (HFIR) SANS instruments. Also, saving a mask that `drtsans` will use still requires a Mantid environment, as described in Section 9.

The USANS workflow in Section 12 has been tested from start to end with data from one proposal. The grouping and scan-count checks use current conventions: consecutive blocks with the same title, a transmission run at the end, and an empty-cell title that names a banjo. These conventions apply to the experiments examined but are not required by the data format. Therefore, the program presents the generated table as a draft for review and does not reduce it automatically.

Current work has two directions: creating digital object identifiers (DOIs) for tagged file sets through the Zenodo API, and adding a natural-language layer that translates a free-text instruction into registry commands for user confirmation. Both would use the existing command interface.

ACKNOWLEDGMENTS

This research used resources at the SNS, a US Department of Energy (DOE) Office of Science User Facility operated by ORNL. Feedback from the first instrument scientist to use the mask editor with production data led to substantial changes in its interaction design.

17. REFERENCES

- Arnold, O., J. C. Bilheux, J. M. Borreguero, et al. 2014. “Mantid—Data Analysis and Visualization Package for Neutron Scattering and μ SR Experiments.” *Nuclear Instruments and Methods in Physics Research Section A* 764:156–166. <https://doi.org/10.1016/j.nima.2014.07.029>.
- Collette, Andrew. 2025. *HDF5 for Python (h5py)*. Software. <https://www.h5py.org/>.
- Do, Changwoo. 2025. *eqsanscli: Command-Line Tools for EQ-SANS*. Software. <https://github.com/cw-do/eqsanscli>.
- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. “Array Programming with NumPy.” *Nature* 585 (7825): 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.
- Huang, Guan-Rong, Chi-Huan Tung, Pengwen Chen, Gernot Rother, Yingrui Shang, Carrie Y. Gao, Michelle Everett, et al. 2026. “Analytical desmearing of Bonse–Hart ultra-small-angle neutron scattering data via truncated Abel inversion.” *Journal of Applied Crystallography* 59:1083–1093. <https://doi.org/10.1107/S1600576726004693>.
- Hunter, John D. 2007. “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering* 9 (3): 90–95. <https://doi.org/10.1109/MCSE.2007.55>.
- Kline, Steven R. 2006. “Reduction and analysis of SANS and USANS data using IGOR Pro.” *Journal of Applied Crystallography* 39 (6): 895–900. <https://doi.org/10.1107/S0021889806035059>.
- Lake, J. A. 1967. “An iterative method of slit-correcting small angle X-ray data.” *Acta Crystallographica* 23 (2): 191–194. <https://doi.org/10.1107/S0365110X67002440>.
- McGugan, Will. 2025. *Textual: a Rapid Application Development Framework for Python*. Software. <https://textual.textualize.io/>.
- NeXus International Advisory Committee. 2025. *The NeXus Data Format*. Standard. <https://www.nexusformat.org/>.
- Oak Ridge National Laboratory. 2025a. *drtsans: Data Reduction Toolkit for Small-Angle Neutron Scattering*. Software. <https://github.com/neutrons/drtsans>.
- . 2025b. *OnCat: Oak Ridge Neutron Catalog*. Web service. <https://oncat.ornl.gov/>.
- Preston-Werner, Tom, et al. 2021. *TOML: Tom’s Obvious Minimal Language*. <https://toml.io/>. Specification v1.0.0.
- Ronacher, Armin, and The Pallets Team. 2025. *Click: Composable Command Line Interface Toolkit*. Software. <https://click.palletsprojects.com/>.
- The Midnight Commander Development Team. 2025. *GNU Midnight Commander*. Software. <https://midnight-commander.org/>.
- Tung, Chi-Huan, Guan-Rong Huang, Yingrui Shang, Gernot Rother, Changwoo Do, Yangyang Wang, Jan-Michael Carrillo, et al. 2026. “Desmearing Bonse–Hart USANS data using Bayesian Gaussian process regression.” *The Journal of Chemical Physics* 164 (20): 204102. <https://doi.org/10.1063/5.0320684>.

Zhao, J. K., C. Y. Gao, and D. Liu. 2010. “The Extended Q-Range Small-Angle Neutron Scattering Diffractometer at the SNS.” *Journal of Applied Crystallography* 43 (5): 1068–1077. <https://doi.org/10.1107/S002188981002217X>.

APPENDIX A. COMMAND REGISTRY REFERENCE

APPENDIX A. COMMAND REGISTRY REFERENCE

Table A-1 lists all commands registered in version 0.10.2. Each command can be entered on the `:` line by its name or alias.

Table A-1. Registered commands. A trailing question mark marks an optional parameter

Command (alias)	Parameters	Description
<code>app.browse_tree</code>	<code>root?</code>	Open a directory tree browser; the selected directory becomes the active pane's.
<code>app.cmdline_open</code>	—	Focus the command line.
<code>app.cmdline_prompt</code>	<code>text</code>	Open the command line pre-filled.
<code>app.help (help)</code>	—	Show the help overlay.
<code>app.quit (quit, q)</code>	—	Exit.
<code>archive.tar_gz (tar)</code>	<code>srcs, out_path</code>	Create a gzip-compressed tar archive.
<code>archive.zip</code>	<code>srcs, out_path</code>	Create a zip archive.
<code>edit.file (edit)</code>	<code>path</code>	Edit in \$EDITOR.
<code>file.copy</code>	<code>srcs, dst_dir</code>	Copy into a destination directory.
<code>file.delete</code>	<code>paths, trash?</code>	Delete or trash.
<code>file.mkdir (mkdir)</code>	<code>name</code>	Create a directory.
<code>file.move</code>	<code>srcs, dst_dir</code>	Move, or rename a single file.
<code>hdf.batch_extract (extract)</code>	<code>paths, keys, out?, fmt?, with_stats?</code>	Extract log values from many NeXus files into a table.
<code>hdf.show_keys (hdf)</code>	<code>path</code>	Open the Hierarchical Data Format version 5 (HDF5) tree browser.
<code>instrument.set (instrument)</code>	<code>name?</code>	Switch the session between small-angle neutron scattering (SANS) and ultra-small-angle neutron scattering (USANS).
<code>mail.send</code>	<code>recipient, subject?, attachments?, body?</code>	Send mail via the configured client.
<code>nav.cd (cd)</code>	<code>path</code>	Change the active pane's directory.
<code>nav.up</code>	—	Ascend one directory.
<code>oncat.search (ipts)</code>	<code>keyword?, instrument?</code>	Browse OnCat experiments.
<code>pane.activate</code>	<code>panel_id</code>	Activate the named pane.
<code>pane.swap</code>	—	Swap the two panes.
<code>pane.sync</code>	—	Match the inactive pane to the active one.
<code>pane.toggle_catalog</code>	—	Flip the inactive pane between file list and run catalog.
<code>pane.toggle_max</code>	—	Maximize or restore the active pane.
<code>plot.detector_sum</code>	<code>paths</code>	Render NeXus detector banks as heat maps.
<code>plot.generic (plot.linear, lplot)</code>	<code>paths</code>	Linear plot of any tabular file.
<code>plot.image (image, view.image)</code>	<code>paths</code>	Open image files in matplotlib.
<code>plot.iq</code>	<code>paths</code>	Overlay 2-, 3-, or 4-column $I(q)$ files.
<code>plot.iqxqy</code>	<code>paths, colorbar_mode?</code>	Plot 4- or 6-column $I(q_x, q_y)$ files; <code>colorbar_mode</code> is shared or independent.
<code>plot.transmission</code>	<code>paths</code>	Plot $T(\lambda)$ on linear axes.
<code>shell.run (!)</code>	<code>cmd</code>	Run a shell command with the interface suspended.
<code>tag.clear</code>	—	Clear all tags in the active pane.

Table A-1, continued. A trailing question mark marks an optional parameter.

Command (alias)	Parameters	Description
tag.glob	pattern	Tag visible entries matching a glob.
tag.toggle	advance?	Toggle the tag on the cursor row.
tag.untag_glob	pattern	Untag entries matching a glob.
ui.activate_cursor	—	Context-sensitive Enter .
ui.batch_extract	paths?	Open the extraction dialog.
ui.copy_tagged	—	Copy the selection to the inactive pane.
ui.delete_tagged	—	Delete the selection, with confirmation.
ui.mail_tagged	—	Mail the selection as attachments.
ui.mask(mask)	path?	Open the mask editor.
ui.move_tagged	—	Move the selection to the inactive pane.
ui.plot_auto	—	Plot the selection, routing by file kind.
usans.init_table (usans-init)	start_run?, out_dir?, data_dir?, ipts?	Build a preliminary USANS setup comma-separated values (CSV) and notes.
usans.reduce (usans-reduce)	path?, output_dir?, data_dir?, logbin?	Reduce a setup CSV, or offer to build one.
usans.desmear (desmear)	paths?, sans?, out_dir?, sigma_y?, ask_for_sans?	Desmear reduced USANS curves by truncated Abel inversion.
view.in_other_pane	path	Show a file in the other pane (open, not toggle).
ui.plot_generic	—	Linear plot of the selection.
ui.refresh(refresh)	—	Re-read both listings.
ui.rename(rename)	—	Rename the file under the cursor.
ui.set_theme(theme)	name?	Switch theme; no argument lists available names.
ui.zip_tagged	—	Prompt for an archive path and zip the selection.
view.file(view)	path	Open a read-only pager.
view.set_filter (filter)	pattern?	Filter the active pane; empty clears.
view.set_sort	key, reverse?	Set the sort key.
view.toggle_hidden	—	Show or hide dotfiles.
view.toggle_ other_pane	—	Show the cursor's file in the other pane.

APPENDIX B. DEFAULT KEY MAP

APPENDIX B. DEFAULT KEY MAP

Table B-1 lists the combined default map shown by the help overlay in USANS mode. The SANS map omits **r** and **d**; the remaining bindings are the same. Ten additional bindings are alternate terminal encodings of the same keys and are not shown in the help overlay.

Table B-1. Default key bindings

Key	Action	Command
Tab	Switch the active pane	pane.activate
Ctrl+U	Swap the two panes	pane.swap
=	Sync the inactive pane	pane.sync
Ctrl+O	Maximize or restore	pane.toggle_max
Enter	Open a directory, image, or text file	ui.activate_cursor
Backspace	Ascend one directory	nav.up
Space	Tag or untag the current row	tag.toggle
+	Tag by glob	app.cmdline_prompt
-	Untag by glob	app.cmdline_prompt
u	Untag all	tag.clear
h	Toggle hidden files	view.toggle_hidden
1 2 3 4	Sort by name, time, size, extension	view.set_sort
F2	Rename	ui.rename
F3	View in the other pane	view.toggle_other_pane
F4	Edit	edit.file
F5	Refresh both panes	ui.refresh
F6	Copy to the other pane	ui.copy_tagged
F7	Move to the other pane	ui.move_tagged
F8	Delete	ui.delete_tagged
F9	Make directory	app.cmdline_prompt
g	Jump to a path	app.cmdline_prompt
G	Browse the filesystem tree	app.browse_tree
/	Filter the active pane	app.cmdline_prompt
c	Toggle the run catalog	pane.toggle_catalog
i	OnCat proposal search	oncat.search
p	Plot the selection	ui.plot_auto
l	Linear plot of the selection	ui.plot_generic
m	HDF5 metadata tree	hdf.show_keys
M	Batch metadata extract	ui.batch_extract
K	Create a detector mask	ui.mask
z	Zip the selection	ui.zip_tagged
e	Email the selection	ui.mail_tagged
:	Open the command line	app.cmdline_open
?	Help overlay	app.help
r	Reduce a USANS setup table (USANS mode only)	usans.reduce
d	Desmear the selected USANS curves (USANS mode only)	usans.desmear
q	Quit	app.quit

APPENDIX C. WORKED EXAMPLE SESSION

APPENDIX C. WORKED EXAMPLE SESSION

The following sequence is an example of a data-triage session and uses most of the program's features. It assumes that the proposal exists and that reduced data are available.

1. Launch with **sansdir**.
2. Press **i** to open the OnCat browser. Press **/** to focus the filter, type a proposal number or a keyword from the title, and press **Enter** to move into the result list. Press **Enter** on the desired proposal and confirm. The active pane moves into the proposal's **shared/** directory and the run catalog loads on the right.
3. Move into the catalog with **Tab**, scroll to a run of interest, and press **p** to view its raw detector image. Confirm the beam center and beam-stop placement, then close the plot window.
4. Press **m** on the same run and then **/** to search for **temperature**. Confirm the sample environment value recorded for the run. Press **Esc** twice to return.
5. Press **Tab** back to the file pane and descend into the reduction output directory.
6. Tag the reduced curves to compare: press **+**, enter a glob such as ***20C*Iq.dat**, and press **Enter**. Press **p** to overlay them on logarithmic axes. Press **u** to clear the tags afterwards.
7. Tag the corresponding transmission files and press **p** again; they are recognized as transmission data and drawn on linear axes.
8. Press **M** to extract metadata across the run series. In the key picker, press **/**, search for the log of interest, press **Space** to select it, and press **Ctrl+S**. Choose summary mode and write the table.
9. Press **l** on the resulting table to plot it directly.
10. Tag the files to share, press **z**, and enter an archive name. Then tag the archive, press **e**, and complete the mail dialog.
11. Press **q** to exit.

